23BDS0139_EDA_PROJECT_PHASE_1

October 16, 2026

EDA PROJECT – PHASES 1, 2 & 3

NAME: Divyam Pandey
REG NO: 23BDS0139
SLOT: A1

0.1 PHASE 1: Data loading, cleaning and exploratory analysis

0.1.1 1. Load dataset

[ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

url = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/Hitters.csv"
df = pd.read_csv(url)
df.head()
```

[ ]:

	AtBat	Hits	HmRun	Runs	RBI	Walks	Years	CAtBat	CHits	CHmRun	CRuns
0	293	66	1	30	29	14	1	293	66	1	30
1	315	81	7	24	38	39	14	3449	835	69	321
2	479	130	18	66	72	76	3	1624	457	63	224
3	496	141	20	65	78	37	11	5628	1575	225	828
4	321	87	10	39	42	30	2	396	101	12	48

	CRBI	CWalks	League	Division	PutOuts	Assists	Errors	Salary	NewLeague
0	29	14	A	E	446	33	20	NaN	A
1	414	375	N	W	632	43	10	475.0	N
2	266	263	A	W	880	82	14	480.0	A
3	838	354	N	E	200	11	3	500.0	N
4	46	33	N	E	805	40	4	91.5	N

[ ]:
```
df.shape
```

[ ]:
```
(322, 20)
```

[ ]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 322 entries, 0 to 321
Data columns (total 20 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   AtBat      322 non-null    int64
 1   Hits       322 non-null    int64
 2   HmRun      322 non-null    int64
 3   Runs       322 non-null    int64
 4   RBI        322 non-null    int64
 5   Walks      322 non-null    int64
 6   Years      322 non-null    int64
 7   CAtBat     322 non-null    int64
 8   CHits      322 non-null    int64
 9   CHmRun     322 non-null    int64
 10  CRuns      322 non-null    int64
 11  CRBI       322 non-null    int64
 12  CWalks     322 non-null    int64
 13  League     322 non-null    str
 14  Division   322 non-null    str
 15  PutOuts    322 non-null    int64
 16  Assists    322 non-null    int64
 17  Errors     322 non-null    int64
 18  Salary     263 non-null    float64
 19  NewLeague  322 non-null    str
dtypes: float64(1), int64(16), str(3)
memory usage: 50.4 KB
```

0.1.2 2. Basic statistical analysis

[ ]:
```
df.describe()
```

[ ]:
```
              AtBat        Hits       HmRun        Runs         RBI       Walks  \
count   322.000000  322.000000  322.000000  322.000000  322.000000  322.000000
mean    380.928571  101.024845   10.770186   50.909938   48.027950   38.742236
std     153.404981   46.454741    8.709037   26.024095   26.166895   21.639327
min      16.000000    1.000000    0.000000    0.000000    0.000000    0.000000
25%     255.250000   64.000000    4.000000   30.250000   28.000000   22.000000
50%     379.500000   96.000000    8.000000   48.000000   44.000000   35.000000
75%     512.000000  137.000000   16.000000   69.000000   64.750000   53.000000
max     687.000000  238.000000   40.000000  130.000000  121.000000  105.000000

              Years       CAtBat        CHits      CHmRun        CRuns  \
count   322.000000    322.00000   322.000000  322.000000   322.000000
mean      7.444099   2648.68323   717.571429   69.490683   358.795031
std       4.926087   2324.20587   654.472627   86.266061   334.105886
min       1.000000     19.00000     4.000000    0.000000     1.000000
```

```
25%      4.000000    816.75000   209.000000   14.000000   100.250000
50%      6.000000   1928.00000   508.000000   37.500000   247.000000
75%     11.000000   3924.25000  1059.250000   90.000000   526.250000
max     24.000000  14053.00000  4256.000000  548.000000  2165.000000

              CRBI       CWalks      PutOuts     Assists      Errors  \
count   322.000000   322.000000   322.000000  322.000000  322.000000
mean    330.118012   260.239130   288.937888  106.913043    8.040373
std     333.219617   267.058085   280.704614  136.854876    6.368359
min       0.000000     0.000000     0.000000    0.000000    0.000000
25%      88.750000    67.250000   109.250000    7.000000    3.000000
50%     220.500000   170.500000   212.000000   39.500000    6.000000
75%     426.250000   339.250000   325.000000  166.000000   11.000000
max    1659.000000  1566.000000  1378.000000  492.000000   32.000000

            Salary
count   263.000000
mean    535.925882
std     451.118681
min      67.500000
25%     190.000000
50%     425.000000
75%     750.000000
max    2460.000000
```

[ ]:
```
df.describe(include='all')
```

[ ]:
```
             AtBat        Hits       HmRun        Runs         RBI  \
count   322.000000  322.000000  322.000000  322.000000  322.000000
unique         NaN         NaN         NaN         NaN         NaN
top            NaN         NaN         NaN         NaN         NaN
freq           NaN         NaN         NaN         NaN         NaN
mean    380.928571  101.024845   10.770186   50.909938   48.027950
std     153.404981   46.454741    8.709037   26.024095   26.166895
min      16.000000    1.000000    0.000000    0.000000    0.000000
25%     255.250000   64.000000    4.000000   30.250000   28.000000
50%     379.500000   96.000000    8.000000   48.000000   44.000000
75%     512.000000  137.000000   16.000000   69.000000   64.750000
max     687.000000  238.000000   40.000000  130.000000  121.000000

             Walks       Years       CAtBat        CHits      CHmRun  \
count   322.000000  322.000000    322.00000   322.000000  322.000000
unique         NaN         NaN          NaN          NaN         NaN
top            NaN         NaN          NaN          NaN         NaN
freq           NaN         NaN          NaN          NaN         NaN
mean     38.742236    7.444099   2648.68323   717.571429   69.490683
std      21.639327    4.926087   2324.20587   654.472627   86.266061
```

```
min       0.000000    1.000000     19.00000     4.000000    0.000000
25%      22.000000    4.000000    816.75000   209.000000   14.000000
50%      35.000000    6.000000   1928.00000   508.000000   37.500000
75%      53.000000   11.000000   3924.25000  1059.250000   90.000000
max     105.000000   24.000000  14053.00000  4256.000000  548.000000

              CRuns         CRBI       CWalks League Division      PutOuts  \
count    322.000000   322.000000   322.000000    322      322   322.000000
unique          NaN          NaN          NaN      2        2          NaN
top             NaN          NaN          NaN      A        W          NaN
freq            NaN          NaN          NaN    175      165          NaN
mean     358.795031   330.118012   260.239130    NaN      NaN   288.937888
std      334.105886   333.219617   267.058085    NaN      NaN   280.704614
min        1.000000     0.000000     0.000000    NaN      NaN     0.000000
25%      100.250000    88.750000    67.250000    NaN      NaN   109.250000
50%      247.000000   220.500000   170.500000    NaN      NaN   212.000000
75%      526.250000   426.250000   339.250000    NaN      NaN   325.000000
max     2165.000000  1659.000000  1566.000000    NaN      NaN  1378.000000

           Assists      Errors       Salary NewLeague
count   322.000000  322.000000   263.000000       322
unique         NaN         NaN          NaN         2
top            NaN         NaN          NaN         A
freq           NaN         NaN          NaN       176
mean    106.913043    8.040373   535.925882       NaN
std     136.854876    6.368359   451.118681       NaN
min       0.000000    0.000000    67.500000       NaN
25%       7.000000    3.000000   190.000000       NaN
50%      39.500000    6.000000   425.000000       NaN
75%     166.000000   11.000000   750.000000       NaN
max     492.000000   32.000000  2460.000000       NaN
```

0.1.3 3. Handling missing data

[ ]:
```
df.isnull().sum()
```

[ ]:
```
AtBat        0
Hits         0
HmRun        0
Runs         0
RBI          0
Walks        0
Years        0
CAtBat       0
CHits        0
CHmRun       0
CRuns        0
```

```
CRBI          0
CWalks        0
League        0
Division      0
PutOuts       0
Assists       0
Errors        0
Salary       59
NewLeague     0
dtype: int64
```

[ ]:
```
df['Salary'] = df['Salary'].fillna(df['Salary'].median())
df.isnull().sum()
```

[ ]:
```
AtBat        0
Hits         0
HmRun        0
Runs         0
RBI          0
Walks        0
Years        0
CAtBat       0
CHits        0
CHmRun       0
CRuns        0
CRBI         0
CWalks       0
League       0
Division     0
PutOuts      0
Assists      0
Errors       0
Salary       0
NewLeague    0
dtype: int64
```

0.1.4 4. Data cleaning

[ ]:
```
df.duplicated().sum()
```

[ ]:
```
np.int64(0)
```

[ ]:
```
df = df.drop_duplicates()
df.columns = df.columns.str.strip()
```

[ ]:
```
df['League'].unique(), df['Division'].unique(), df['NewLeague'].unique()
```

[ ]:
```
(<StringArray>
 ['A', 'N']
 Length: 2, dtype: str,
 <StringArray>
 ['E', 'W']
 Length: 2, dtype: str,
 <StringArray>
 ['A', 'N']
 Length: 2, dtype: str)
```

0.1.5 5. Data transformation

[ ]:
```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['League'] = le.fit_transform(df['League'])
df['Division'] = le.fit_transform(df['Division'])
df['NewLeague'] = le.fit_transform(df['NewLeague'])

df['Salary_log'] = np.log(df['Salary'] + 1)
df.head()
```

[ ]:

	AtBat	Hits	HmRun	Runs	RBI	Walks	Years	CAtBat	CHits	CHmRun	…	\
0	293	66	1	30	29	14	1	293	66	1	…	
1	315	81	7	24	38	39	14	3449	835	69	…	
2	479	130	18	66	72	76	3	1624	457	63	…	
3	496	141	20	65	78	37	11	5628	1575	225	…	
4	321	87	10	39	42	30	2	396	101	12	…	

	CRBI	CWalks	League	Division	PutOuts	Assists	Errors	Salary	\
0	29	14	0	0	446	33	20	425.0	
1	414	375	1	1	632	43	10	475.0	
2	266	263	0	1	880	82	14	480.0	
3	838	354	1	0	200	11	3	500.0	
4	46	33	1	0	805	40	4	91.5	

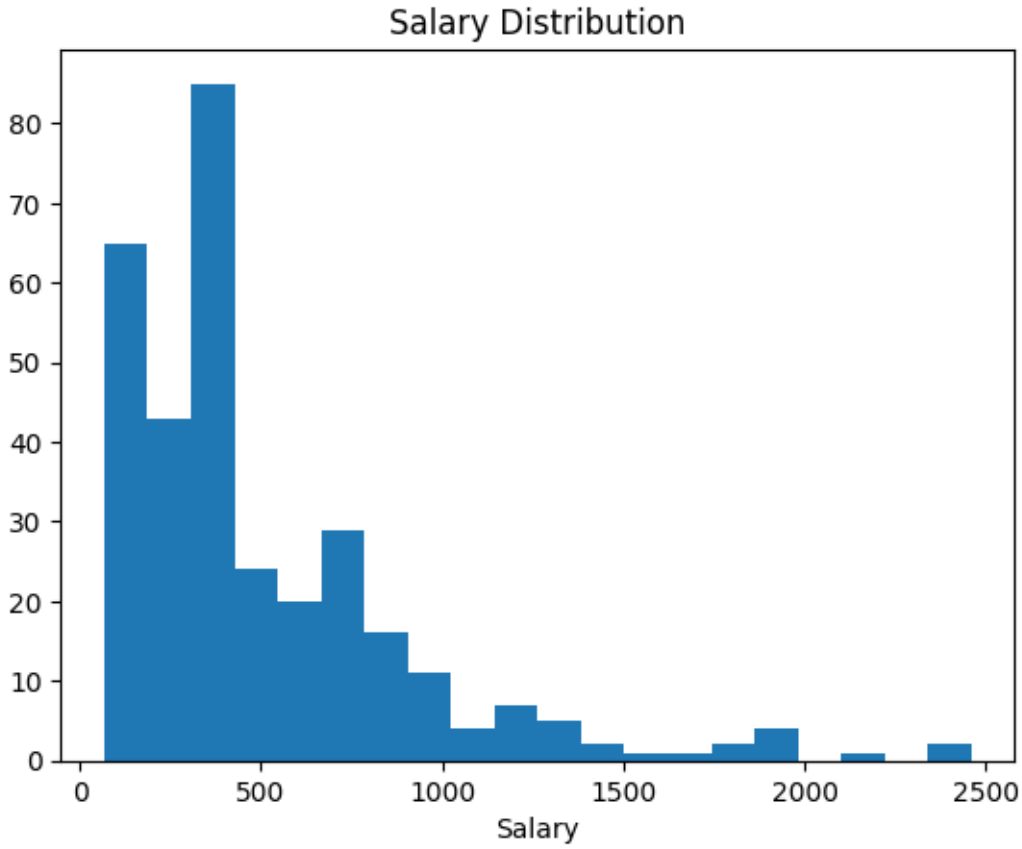
	NewLeague	Salary_log
0	0	6.054439
1	1	6.165418
2	0	6.175867
3	1	6.216606
4	1	4.527209

[5 rows x 21 columns]

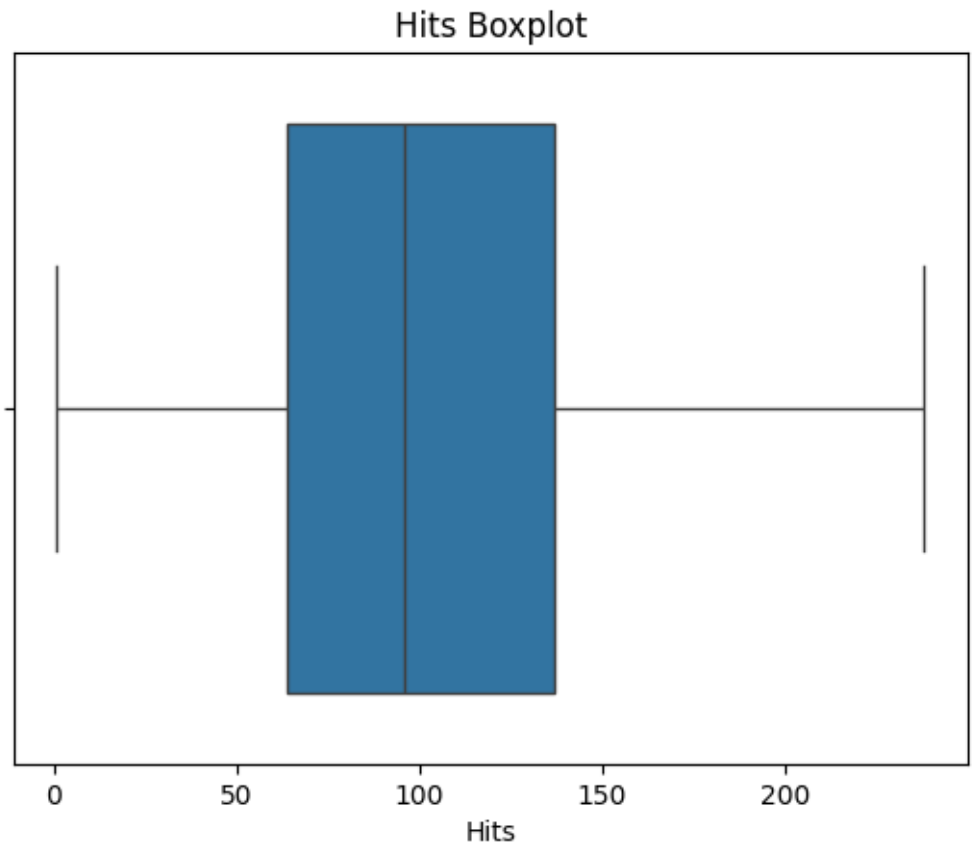
0.1.6 6. Univariate (1D) Analysis

Quick distribution plots first, then a full statistical breakdown with central tendency, dispersion, shape metrics, and outlier detection.

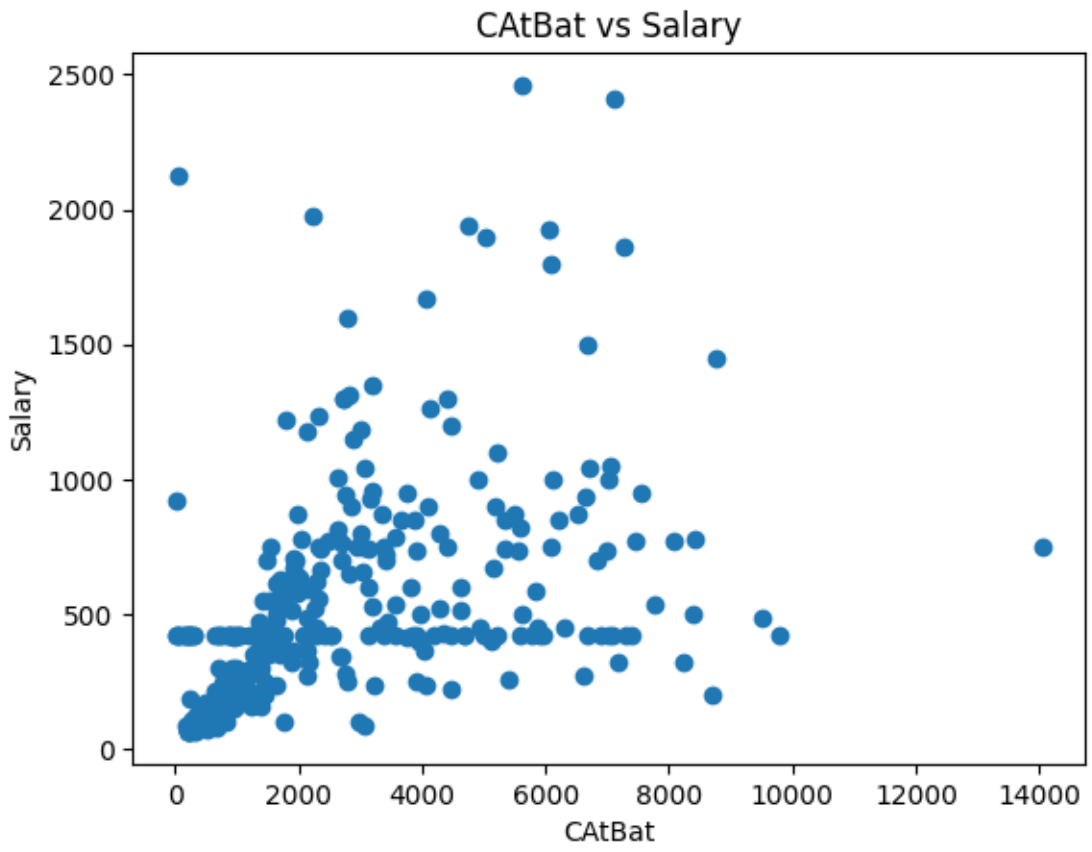
[ ]:
```
plt.hist(df['Salary'], bins=20)
plt.title("Salary Distribution")
plt.xlabel("Salary")
plt.show()
```



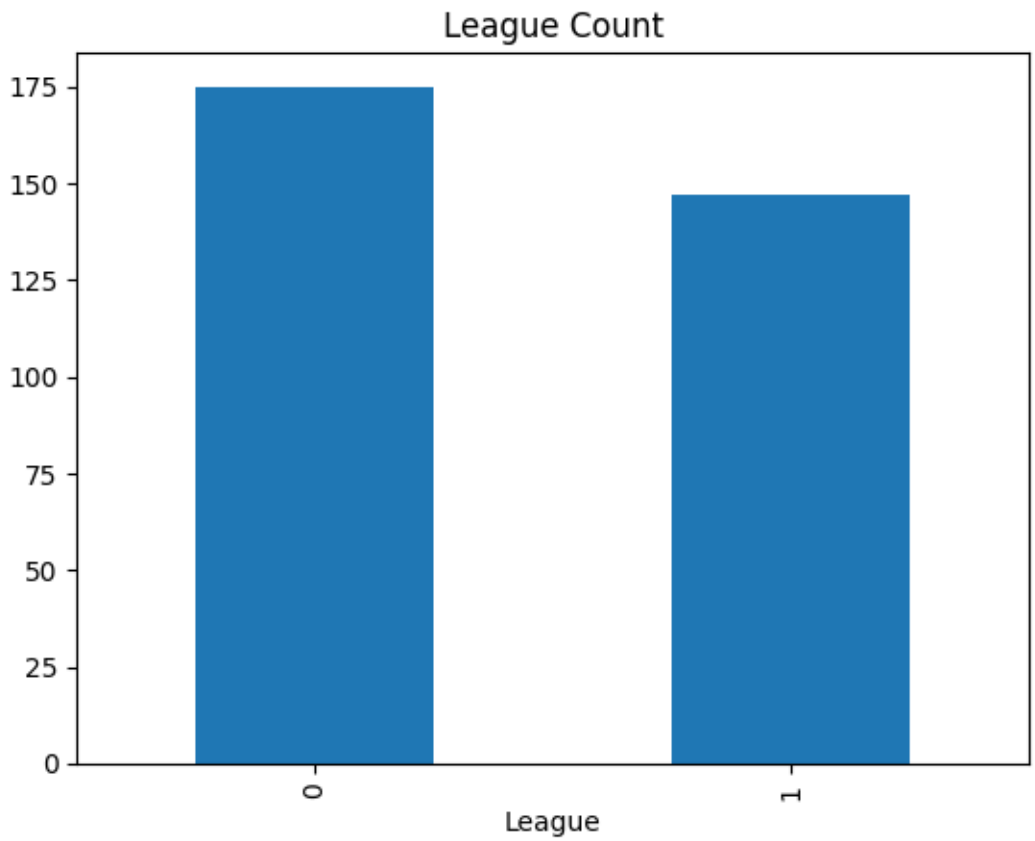
[ ]:
```
sns.boxplot(x=df['Hits'])
plt.title("Hits Boxplot")
plt.show()
```



[ ]:
```
plt.scatter(df['CAtBat'], df['Salary'])
plt.xlabel("CAtBat")
plt.ylabel("Salary")
plt.title("CAtBat vs Salary")
plt.show()
```



[ ]:
```
df['League'].value_counts().plot(kind='bar')
plt.title("League Count")
plt.show()
```



6.1 Detailed numeric summaries

[ ]:
```
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_like = ['League', 'Division', 'NewLeague']
numeric_cols_for_stats = [c for c in numeric_cols if c not in categorical_like]
numeric_cols_for_stats
```

[ ]:
```
['AtBat',
 'Hits',
 'HmRun',
 'Runs',
 'RBI',
 'Walks',
 'Years',
 'CAtBat',
 'CHits',
 'CHmRun',
 'CRuns',
 'CRBI',
 'CWalks',
 'PutOuts',
```

```
 'Assists',
 'Errors',
 'Salary',
 'Salary_log']
```

[ ]:
```
def univariate_summary(series):
    return pd.Series({
        'count': series.count(),
        'mean': series.mean(),
        'median': series.median(),
        'mode': series.mode().iloc[0] if not series.mode().empty else np.nan,
        'std': series.std(),
        'variance': series.var(),
        'min': series.min(),
        'max': series.max(),
        'range': series.max() - series.min(),
        'Q1': series.quantile(0.25),
        'Q3': series.quantile(0.75),
        'IQR': series.quantile(0.75) - series.quantile(0.25),
        'skewness': series.skew(),
        'kurtosis': series.kurt()
    })

univariate_stats = pd.DataFrame({col: univariate_summary(df[col]) for col in␣
 ↪numeric_cols_for_stats}).T
univariate_stats
```

[ ]:

	count	mean	median	mode	std	\
AtBat	322.0	380.928571	379.500000	209.000000	153.404981	
Hits	322.0	101.024845	96.000000	53.000000	46.454741	
HmRun	322.0	10.770186	8.000000	4.000000	8.709037	
Runs	322.0	50.909938	48.000000	42.000000	26.024095	
RBI	322.0	48.027950	44.000000	29.000000	26.166895	
Walks	322.0	38.742236	35.000000	22.000000	21.639327	
Years	322.0	7.444099	6.000000	4.000000	4.926087	
CAtBat	322.0	2648.683230	1928.000000	41.000000	2324.205870	
CHits	322.0	717.571429	508.000000	160.000000	654.472627	
CHmRun	322.0	69.490683	37.500000	32.000000	86.266061	
CRuns	322.0	358.795031	247.000000	20.000000	334.105886	
CRBI	322.0	330.118012	220.500000	32.000000	333.219617	
CWalks	322.0	260.239130	170.500000	55.000000	267.058085	
PutOuts	322.0	288.937888	212.000000	0.000000	280.704614	
Assists	322.0	106.913043	39.500000	0.000000	136.854876	
Errors	322.0	8.040373	6.000000	3.000000	6.368359	
Salary	322.0	515.600953	425.000000	425.000000	409.817459	
Salary_log	322.0	5.953735	6.054439	6.054439	0.801759	

	variance	min	max	range	Q1	\
AtBat	2.353309e+04	16.000000	687.000000	671.000000	255.250000	
Hits	2.158043e+03	1.000000	238.000000	237.000000	64.000000	
HmRun	7.584733e+01	0.000000	40.000000	40.000000	4.000000	
Runs	6.772535e+02	0.000000	130.000000	130.000000	30.250000	
RBI	6.847064e+02	0.000000	121.000000	121.000000	28.000000	
Walks	4.682605e+02	0.000000	105.000000	105.000000	22.000000	
Years	2.426634e+01	1.000000	24.000000	23.000000	4.000000	
CAtBat	5.401933e+06	19.000000	14053.000000	14034.000000	816.750000	
CHits	4.283344e+05	4.000000	4256.000000	4252.000000	209.000000	
CHmRun	7.441833e+03	0.000000	548.000000	548.000000	14.000000	
CRuns	1.116267e+05	1.000000	2165.000000	2164.000000	100.250000	
CRBI	1.110353e+05	0.000000	1659.000000	1659.000000	88.750000	
CWalks	7.132002e+04	0.000000	1566.000000	1566.000000	67.250000	
PutOuts	7.879508e+04	0.000000	1378.000000	1378.000000	109.250000	
Assists	1.872926e+04	0.000000	492.000000	492.000000	7.000000	
Errors	4.055600e+01	0.000000	32.000000	32.000000	3.000000	
Salary	1.679503e+05	67.500000	2460.000000	2392.500000	226.250000	
Salary_log	6.428183e-01	4.226834	7.808323	3.581489	5.426006	

	Q3	IQR	skewness	kurtosis
AtBat	512.000000	256.750000	-0.078061	-0.889432
Hits	137.000000	73.000000	0.291154	-0.502752
HmRun	16.000000	12.000000	0.904803	0.039869
Runs	69.000000	38.750000	0.415779	-0.518264
RBI	64.750000	36.750000	0.608377	-0.300722
Walks	53.000000	31.000000	0.620286	-0.257561
Years	11.000000	7.000000	0.779707	-0.146659
CAtBat	3924.250000	3107.500000	1.227169	1.502198
CHits	1059.250000	850.250000	1.356461	2.354018
CHmRun	90.000000	76.000000	2.163167	5.324175
CRuns	526.250000	426.000000	1.391805	2.397464
CRBI	426.250000	337.500000	1.532064	1.955240
CWalks	339.250000	272.000000	1.752989	3.488463
PutOuts	325.000000	215.750000	2.064147	4.215549
Assists	166.000000	159.000000	1.358438	0.683007
Errors	11.000000	8.000000	1.055453	0.688192
Salary	700.000000	473.750000	1.873611	4.605924
Salary_log	6.552508	1.126502	-0.279314	-0.396881

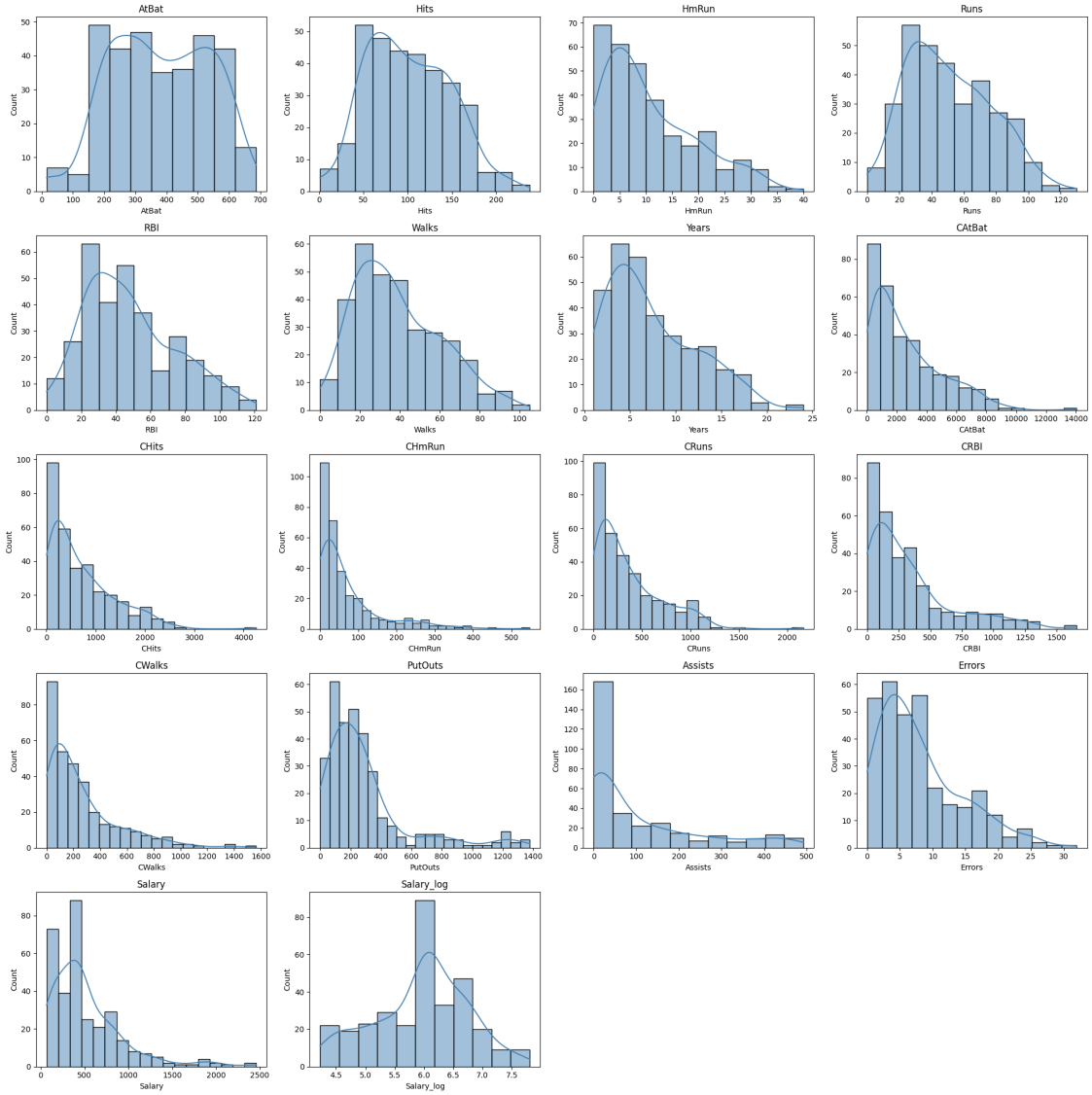
[ ]:
```
for col in numeric_cols_for_stats:
    sk = df[col].skew()
    if sk > 1:
        tag = 'highly right-skewed'
    elif sk > 0.5:
        tag = 'moderately right-skewed'
    elif sk < -1:
```

```
        tag = 'highly left-skewed'
    elif sk < -0.5:
        tag = 'moderately left-skewed'
    else:
        tag = 'approximately symmetric'
    print(f"{col:10s} skew={sk:6.2f}  -> {tag}")
```

```
AtBat      skew= -0.08  -> approximately symmetric
Hits       skew=  0.29  -> approximately symmetric
HmRun      skew=  0.90  -> moderately right-skewed
Runs       skew=  0.42  -> approximately symmetric
RBI        skew=  0.61  -> moderately right-skewed
Walks      skew=  0.62  -> moderately right-skewed
Years      skew=  0.78  -> moderately right-skewed
CAtBat     skew=  1.23  -> highly right-skewed
CHits      skew=  1.36  -> highly right-skewed
CHmRun     skew=  2.16  -> highly right-skewed
CRuns      skew=  1.39  -> highly right-skewed
CRBI       skew=  1.53  -> highly right-skewed
CWalks     skew=  1.75  -> highly right-skewed
PutOuts    skew=  2.06  -> highly right-skewed
Assists    skew=  1.36  -> highly right-skewed
Errors     skew=  1.06  -> highly right-skewed
Salary     skew=  1.87  -> highly right-skewed
Salary_log skew= -0.28  -> approximately symmetric
```

6.2 Distribution plots (histogram + KDE) and boxplots

[ ]:
```
n = len(numeric_cols_for_stats)
ncols = 4
nrows = int(np.ceil(n / ncols))
fig, axes = plt.subplots(nrows, ncols, figsize=(20, 4 * nrows))
axes = axes.flatten()
for i, col in enumerate(numeric_cols_for_stats):
    sns.histplot(df[col], kde=True, ax=axes[i], color='steelblue')
    axes[i].set_title(col)
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()
```



6.3 Outlier counts and categorical frequencies

[ ]:
```
outlier_counts = {}
for col in numeric_cols_for_stats:
    Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    outlier_counts[col] = ((df[col] < lower) | (df[col] > upper)).sum()

pd.Series(outlier_counts, name='outlier_count').sort_values(ascending=False)
```

[ ]:
```
PutOuts      36
CHmRun       32
```

```
CRBI          26
Assists       23
CWalks        20
Salary        13
Errors         9
CHits          6
CRuns          5
CAtBat         5
HmRun          2
Years          2
RBI            1
Runs           1
Walks          1
AtBat          0
Hits           0
Salary_log     0
Name: outlier_count, dtype: int64
```

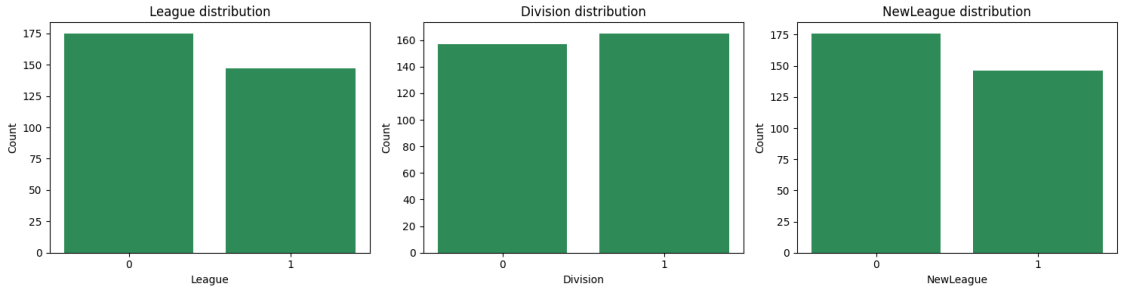
[ ]:
```
categorical_original = {
    'League': le.inverse_transform(df['League']) if False else df['League'],
    'Division': df['Division'],
    'NewLeague': df['NewLeague']
}

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, col in zip(axes, ['League', 'Division', 'NewLeague']):
    counts = df[col].value_counts().sort_index()
    print(f"--- {col} frequency table ---")
    print(counts, "\n")
    ax.bar(counts.index.astype(str), counts.values, color='seagreen')
    ax.set_title(f"{col} distribution")
    ax.set_xlabel(col)
    ax.set_ylabel("Count")
plt.tight_layout()
plt.show()
```

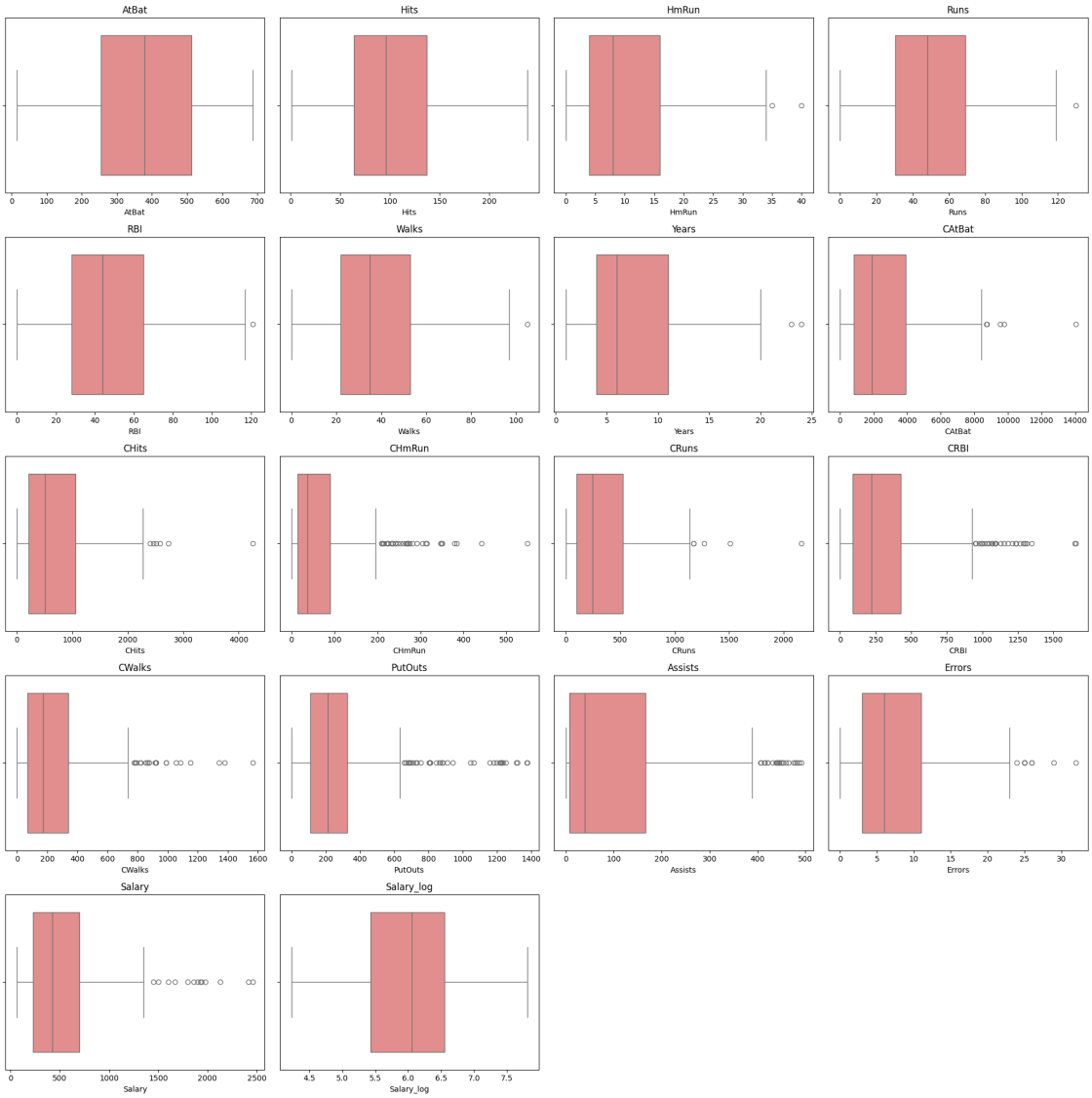
```
--- League frequency table ---
League
0    175
1    147
Name: count, dtype: int64

--- Division frequency table ---
Division
0    157
1    165
Name: count, dtype: int64
```

```
--- NewLeague frequency table ---
NewLeague
0    176
1    146
Name: count, dtype: int64
```



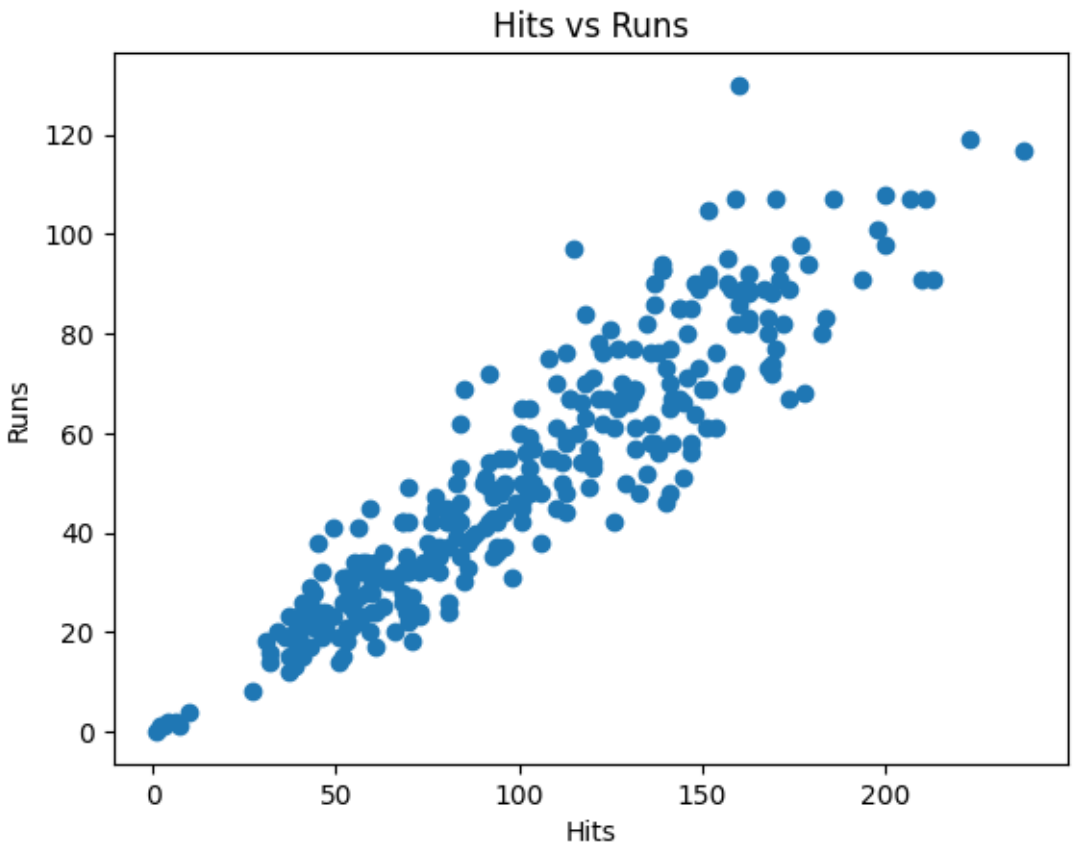
[ ]:
```
fig, axes = plt.subplots(nrows, ncols, figsize=(20, 4 * nrows))
axes = axes.flatten()
for i, col in enumerate(numeric_cols_for_stats):
    sns.boxplot(x=df[col], ax=axes[i], color='lightcoral')
    axes[i].set_title(col)
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()
```



0.1.7 7. Bivariate (2D) Analysis

Starting with simple scatter plots, then going deeper with Pearson & Spearman correlations, regression plots, group comparisons (t-test, ANOVA), and chi-square tests for categorical pairs.

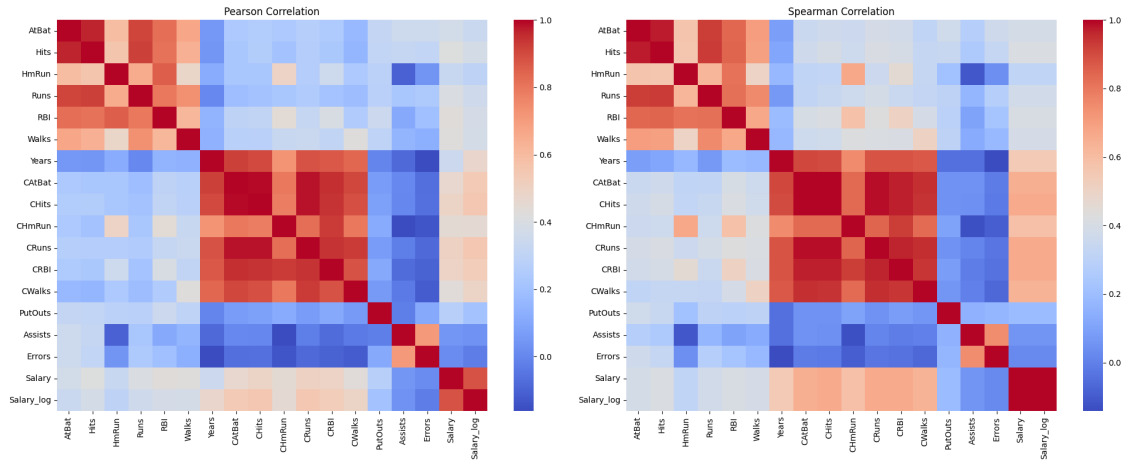
[ ]:
```
plt.scatter(df['Hits'], df['Runs'])
plt.xlabel("Hits")
plt.ylabel("Runs")
plt.title("Hits vs Runs")
plt.show()
```



7.1 Correlation matrices

[ ]:
```
pearson_corr = df[numeric_cols_for_stats].corr(method='pearson')
spearman_corr = df[numeric_cols_for_stats].corr(method='spearman')

fig, axes = plt.subplots(1, 2, figsize=(20, 8))
sns.heatmap(pearson_corr, annot=False, cmap='coolwarm', ax=axes[0])
axes[0].set_title("Pearson Correlation")
sns.heatmap(spearman_corr, annot=False, cmap='coolwarm', ax=axes[1])
axes[1].set_title("Spearman Correlation")
plt.tight_layout()
plt.show()
```



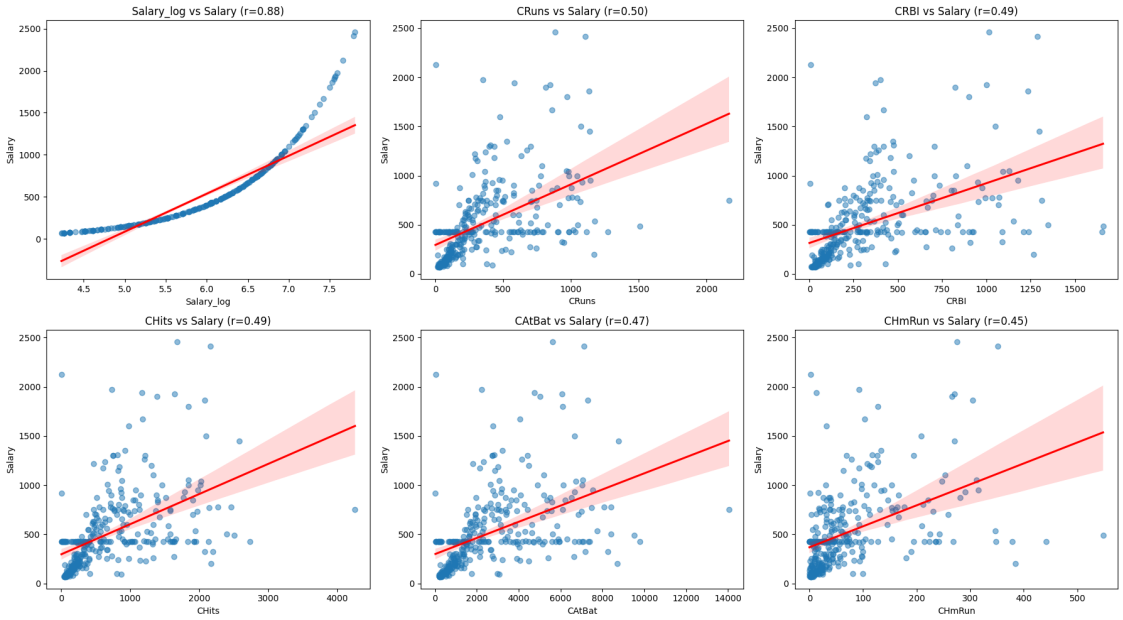
[ ]:
```
salary_corr = pearson_corr['Salary'].drop('Salary').sort_values(key=abs,␣
↪ascending=False)
salary_corr
```

[ ]:
```
Salary_log    0.883243
CRuns         0.502752
CRBI          0.494533
CHits         0.490008
CAtBat        0.465903
CHmRun        0.448738
CWalks        0.435158
RBI           0.428745
Walks         0.424553
Hits          0.415332
Runs          0.402887
AtBat         0.373421
Years         0.344336
HmRun         0.331530
PutOuts       0.270629
Assists       0.043425
Errors        0.014216
Name: Salary, dtype: float64
```

7.2 Scatter + regression plots for top correlated features

[ ]:
```
top_features = salary_corr.abs().sort_values(ascending=False).head(6).index.
↪tolist()
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()
for i, col in enumerate(top_features):
    sns.regplot(x=df[col], y=df['Salary'], ax=axes[i],
                scatter_kws={'alpha': 0.5}, line_kws={'color': 'red'})
```

```
    r = pearson_corr.loc[col, 'Salary']
    axes[i].set_title(f"{col} vs Salary (r={r:.2f})")
plt.tight_layout()
plt.show()
```



7.3 Categorical vs numeric comparisons

[ ]:
```
print("Mean Salary by Division:")
print(df.groupby('Division')['Salary'].mean(), "\n")

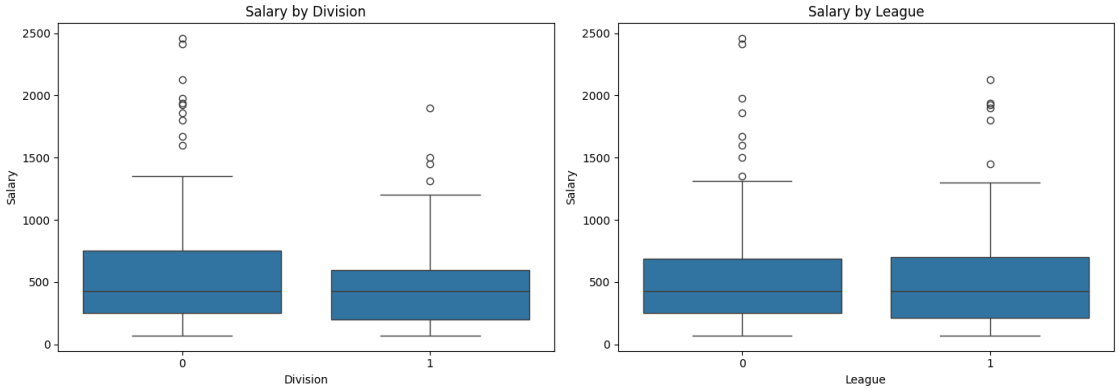
print("Mean Salary by League:")
print(df.groupby('League')['Salary'].mean())
```

```
Mean Salary by Division:
Division
0    588.732522
1    446.015158
Name: Salary, dtype: float64

Mean Salary by League:
League
0    517.931069
1    512.827007
Name: Salary, dtype: float64
```

[ ]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.boxplot(x='Division', y='Salary', data=df, ax=axes[0])
axes[0].set_title("Salary by Division")
```

```
sns.boxplot(x='League', y='Salary', data=df, ax=axes[1])
axes[1].set_title("Salary by League")
plt.tight_layout()
plt.show()
```



[ ]:
```
group0 = df[df['Division'] == 0]['Salary']
group1 = df[df['Division'] == 1]['Salary']
t_stat, p_val = stats.ttest_ind(group0, group1, equal_var=False)
print(f"t-statistic = {t_stat:.3f}, p-value = {p_val:.3f}")
print("Significant difference" if p_val < 0.05 else "No significant␣
 ↪difference", "at alpha=0.05")
```

```
t-statistic = 3.134, p-value = 0.002
Significant difference at alpha=0.05
```

[ ]:
```
groups = [df[df['League'] == lvl]['Salary'] for lvl in df['League'].unique()]
f_stat, p_val = stats.f_oneway(*groups)
print(f"F-statistic = {f_stat:.3f}, p-value = {p_val:.3f}")
```

```
F-statistic = 0.012, p-value = 0.912
```

7.4 Categorical - categorical association

[ ]:
```
contingency = pd.crosstab(df['League'], df['Division'])
print(contingency)

chi2, p, dof, expected = stats.chi2_contingency(contingency)
print(f"\nChi-square = {chi2:.3f}, p-value = {p:.3f}, dof = {dof}")
```

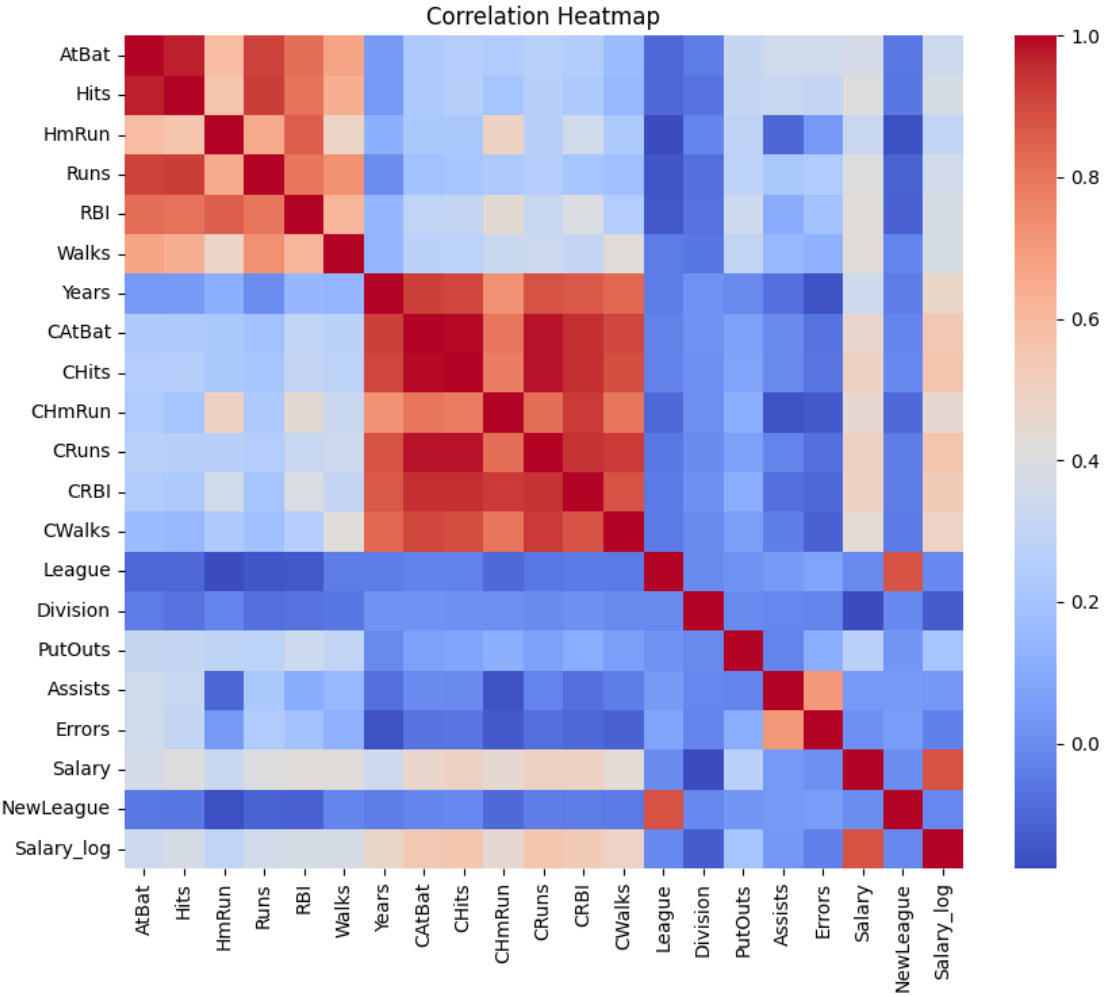
```
Division   0   1
League
0         85  90
1         72  75

Chi-square = 0.000, p-value = 1.000, dof = 1
```

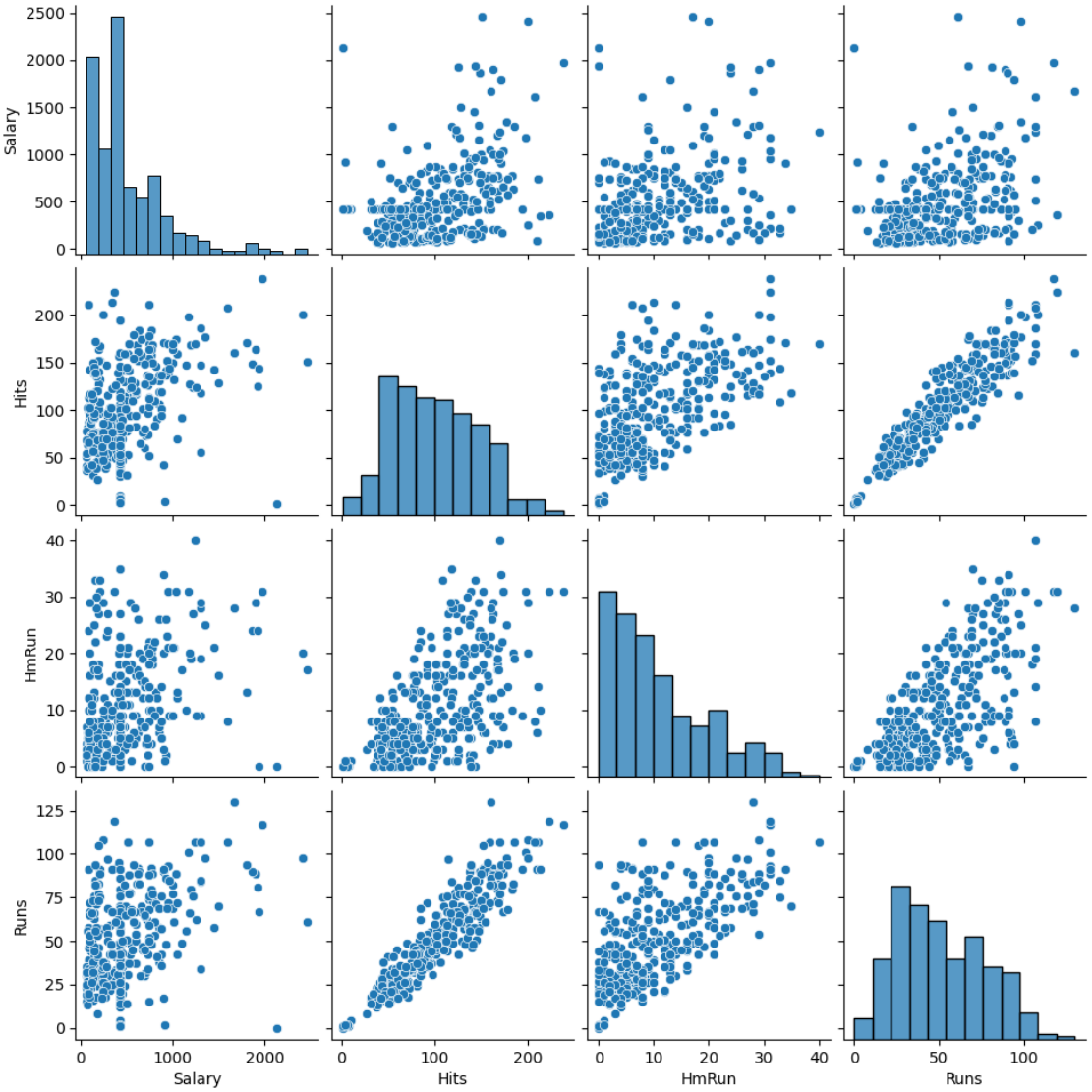
0.1.8 8. Multivariate (3D+) Analysis

Heatmaps, pairplots, 3D scatters, and bubble charts to explore interactions across three or more variables simultaneously.

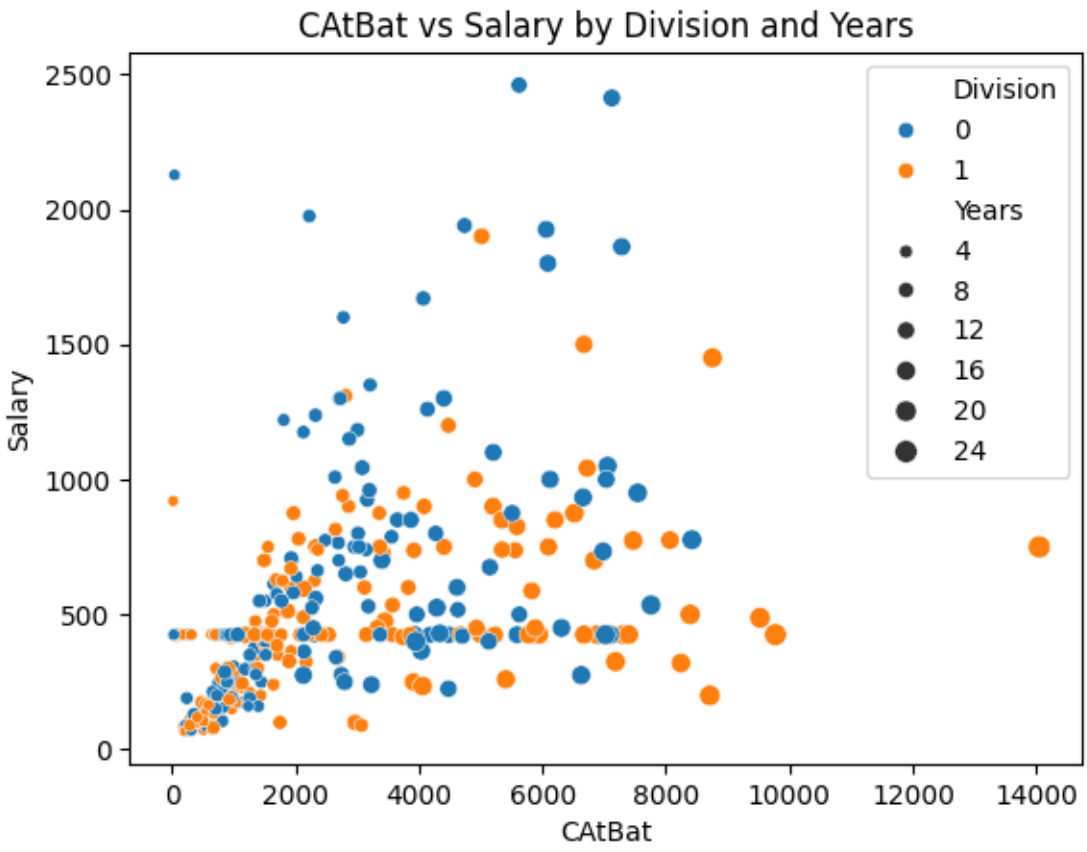
[ ]:
```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```



[ ]:
```
sns.pairplot(df[['Salary', 'Hits', 'HmRun', 'Runs']])
plt.show()
```



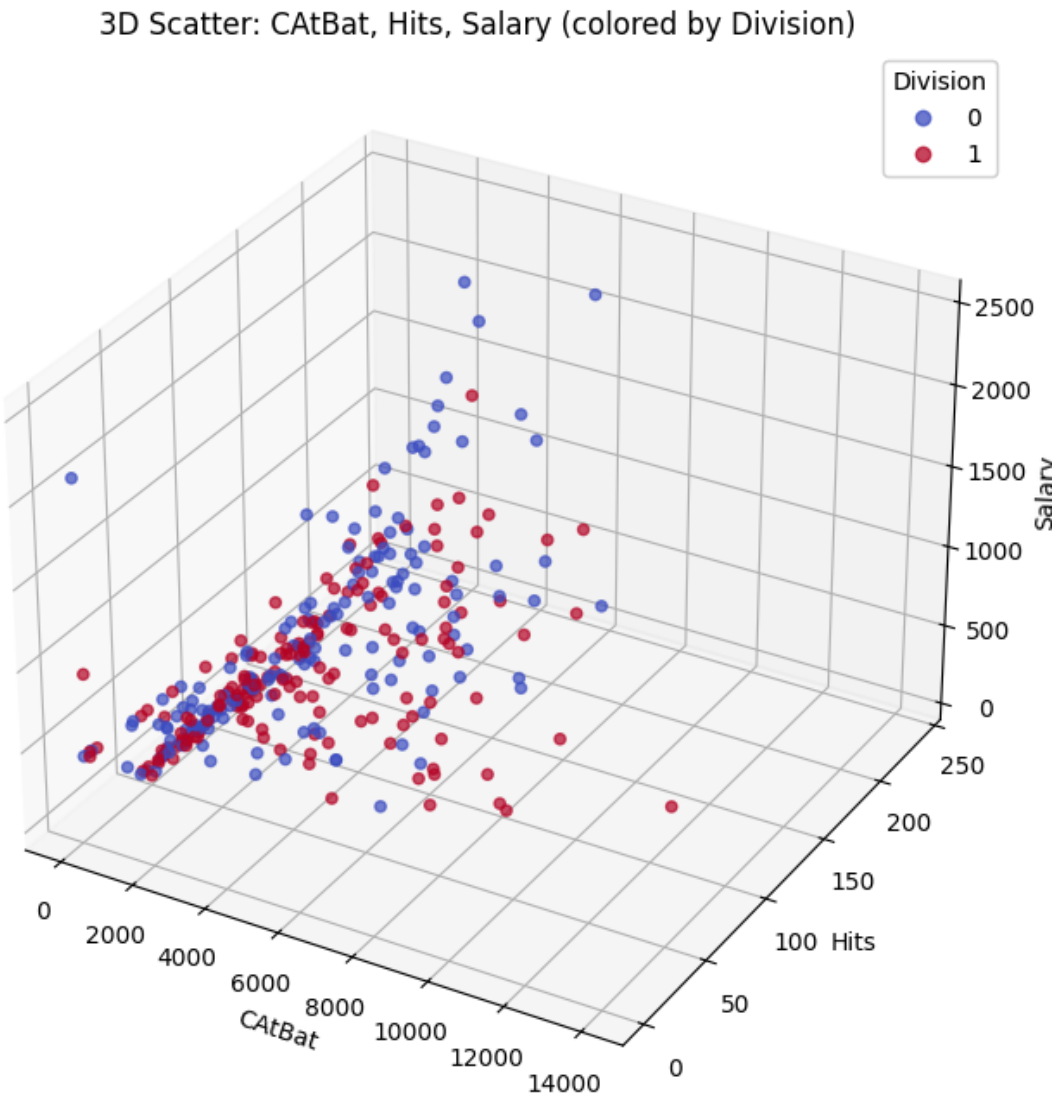
[ ]:
```
sns.scatterplot(x='CAtBat', y='Salary', hue='Division', size='Years', data=df)
plt.title("CAtBat vs Salary by Division and Years")
plt.show()
```



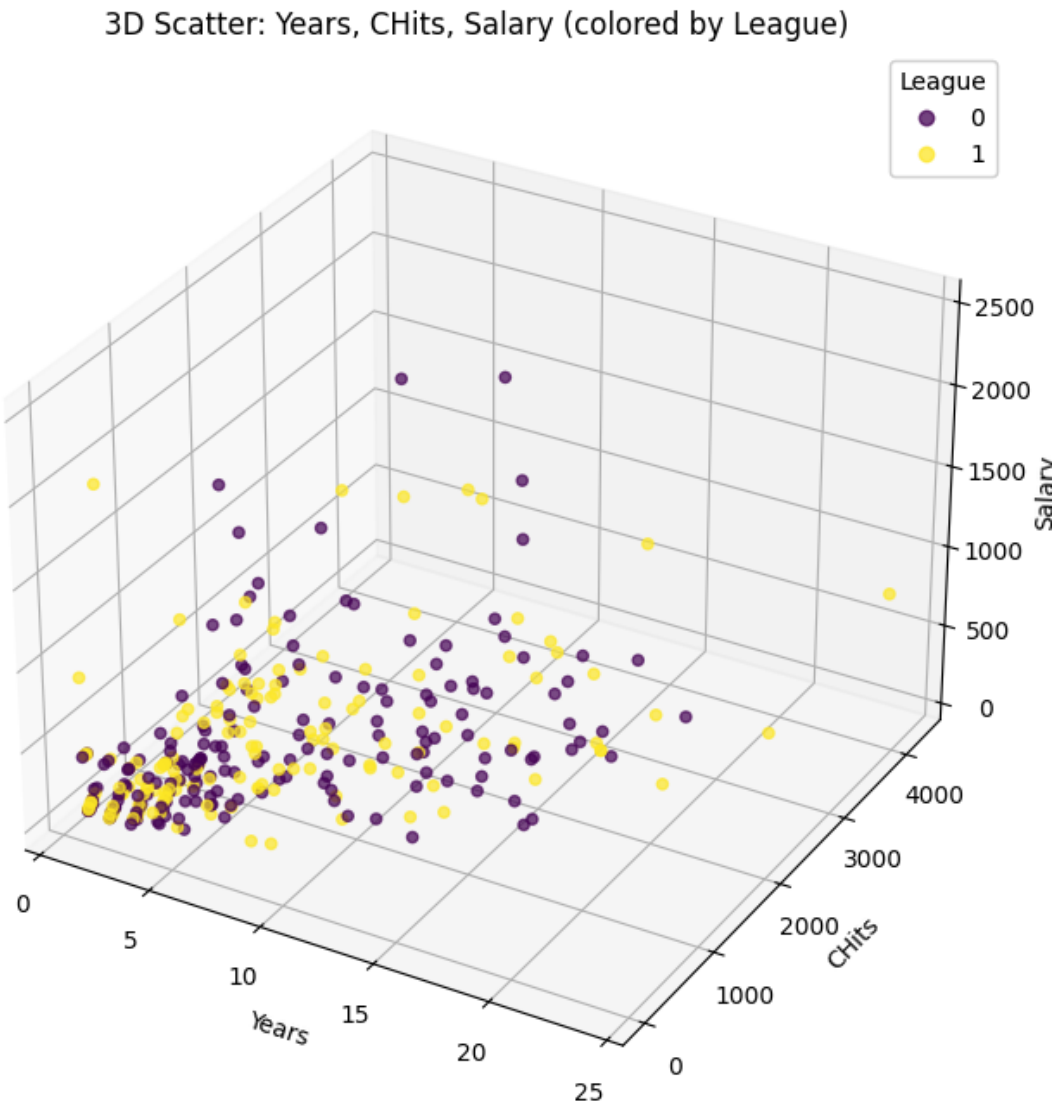
8.1 3D scatter plots

[ ]:
```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(df['CAtBat'], df['Hits'], df['Salary'],
                     c=df['Division'], cmap='coolwarm', alpha=0.7)
ax.set_xlabel('CAtBat')
ax.set_ylabel('Hits')
ax.set_zlabel('Salary')
ax.set_title('3D Scatter: CAtBat, Hits, Salary (colored by Division)')
legend1 = ax.legend(*scatter.legend_elements(), title="Division")
ax.add_artist(legend1)
plt.show()
```

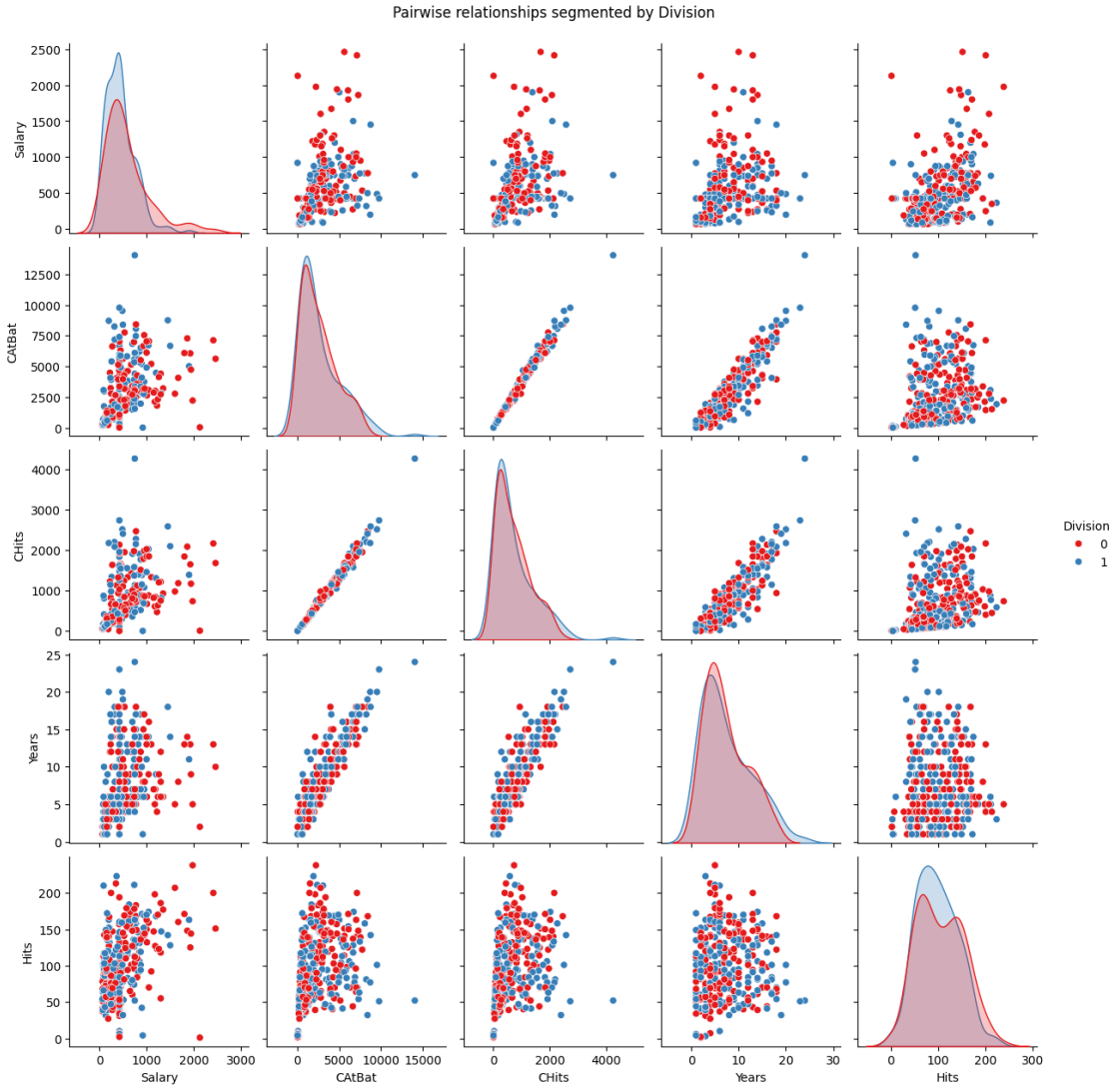


[ ]:
```
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(df['Years'], df['CHits'], df['Salary'],
                     c=df['League'], cmap='viridis', alpha=0.7)
ax.set_xlabel('Years')
ax.set_ylabel('CHits')
ax.set_zlabel('Salary')
ax.set_title('3D Scatter: Years, CHits, Salary (colored by League)')
legend1 = ax.legend(*scatter.legend_elements(), title="League")
ax.add_artist(legend1)
plt.show()
```

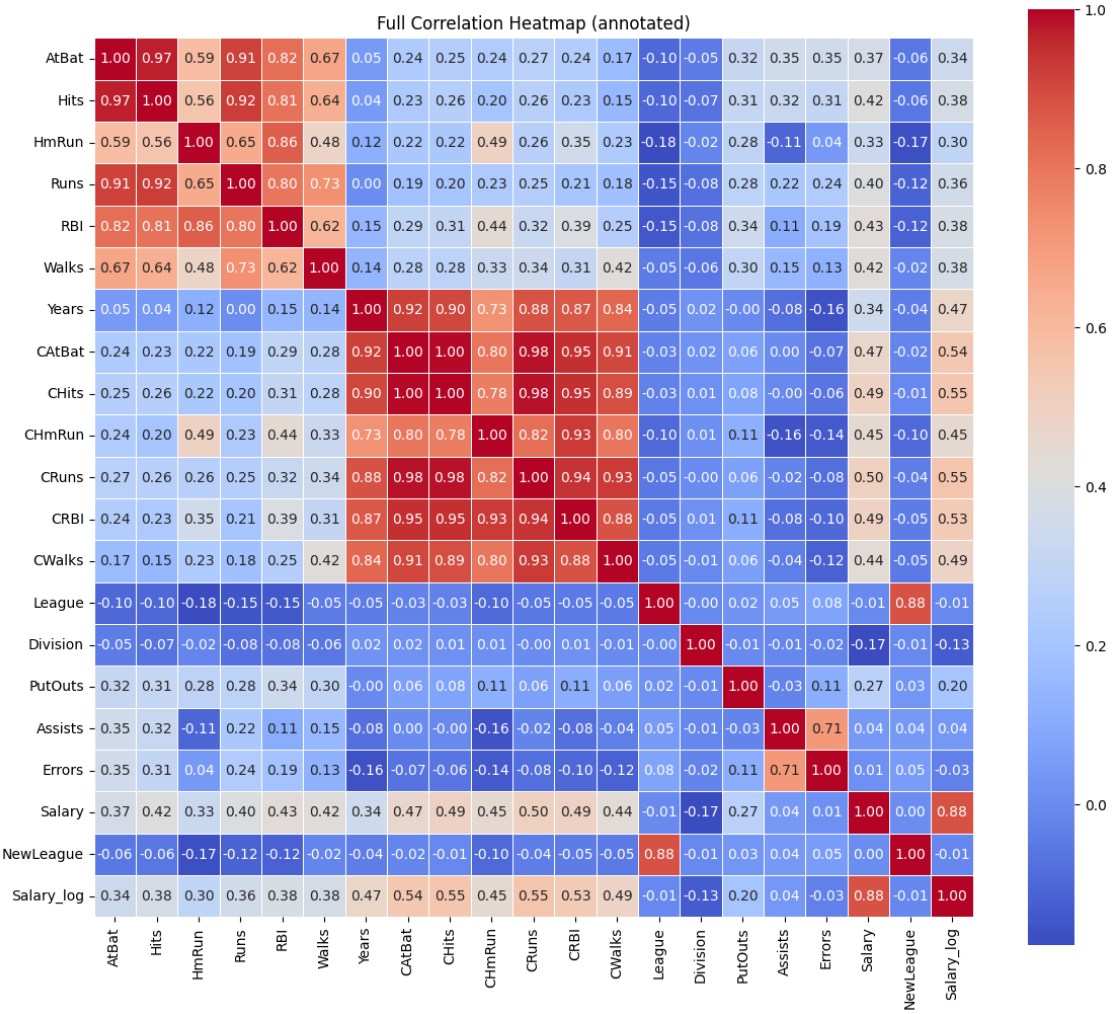


8.2 Segmented pairplot and full annotated heatmap

[ ]:
```
key_vars = ['Salary', 'CAtBat', 'CHits', 'Years', 'Hits']
sns.pairplot(df[key_vars + ['Division']], hue='Division', diag_kind='kde',␣
↪palette='Set1')
plt.suptitle("Pairwise relationships segmented by Division", y=1.02)
plt.show()
```

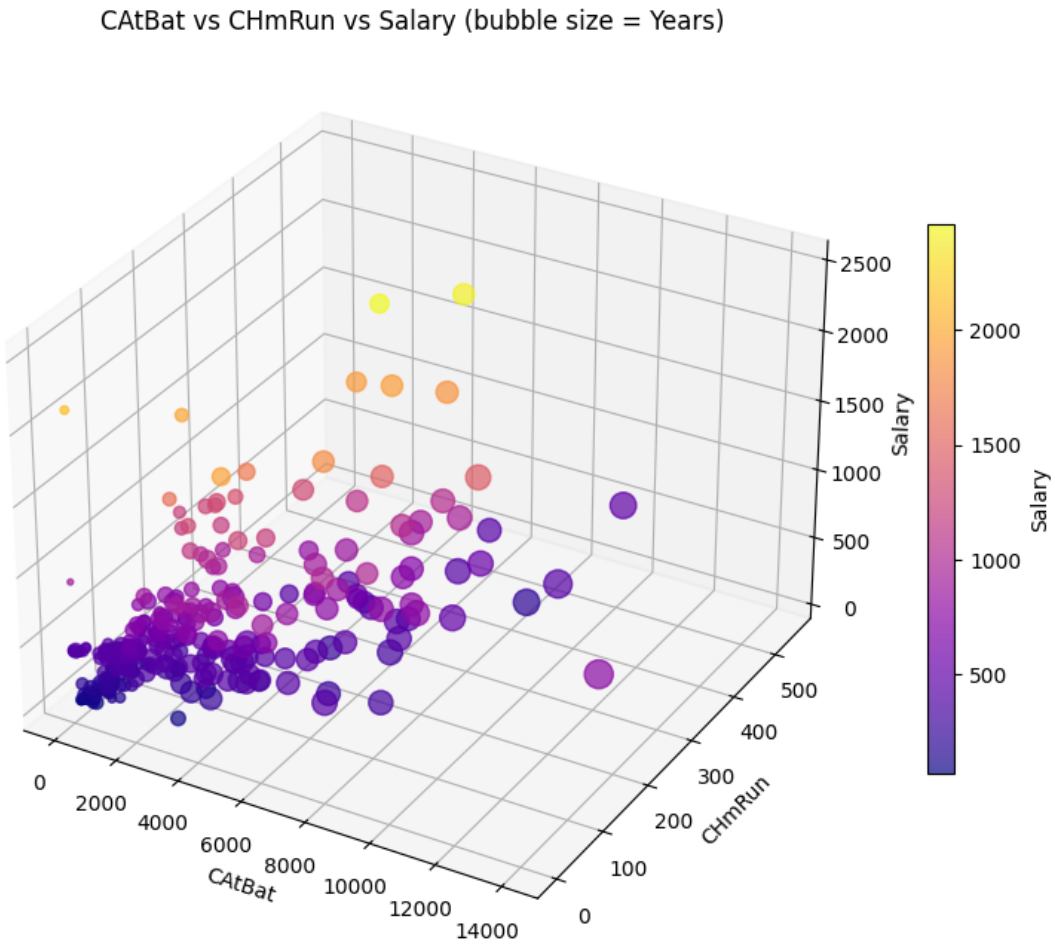


[ ]:
```
plt.figure(figsize=(14, 12))
sns.heatmap(df[numeric_cols].corr(), annot=True, fmt='.2f', cmap='coolwarm',
            square=True, linewidths=0.5)
plt.title("Full Correlation Heatmap (annotated)")
plt.show()
```



8.3 Bubble chart (4th dimension via size)

[ ]:
```
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
sc = ax.scatter(df['CAtBat'], df['CHmRun'], df['Salary'],
                s=df['Years'] * 8, c=df['Salary'], cmap='plasma', alpha=0.7)
ax.set_xlabel('CAtBat')
ax.set_ylabel('CHmRun')
ax.set_zlabel('Salary')
ax.set_title('CAtBat vs CHmRun vs Salary (bubble size = Years)')
fig.colorbar(sc, ax=ax, shrink=0.6, label='Salary')
plt.show()
```



0.2 PHASE 2: Clustering

(The 1D, 2D and 3D analyses are Sections 6 to 8.)

0.2.1 9. K-Means Clustering

Standardizing numeric features, picking k via elbow + silhouette, then fitting K-Means and projecting into 2D with PCA.

[ ]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

cluster_features = ['AtBat', 'Hits', 'HmRun', 'Runs', 'RBI', 'Walks',
                    'Years', 'CAtBat', 'CHits', 'CHmRun', 'CRuns', 'CRBI',
                    'CWalks', 'PutOuts', 'Assists', 'Errors', 'Salary']
```

```
X = df[cluster_features].copy()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled.shape
```

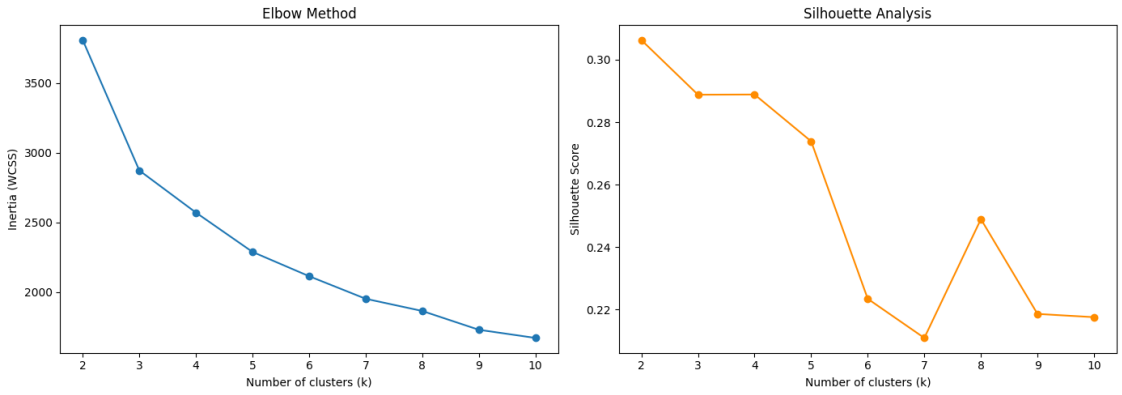
[ ]: (322, 17)

[ ]:
```
inertias = []
sil_scores = []
K_range = range(2, 11)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(list(K_range), inertias, marker='o')
axes[0].set_xlabel('Number of clusters (k)')
axes[0].set_ylabel('Inertia (WCSS)')
axes[0].set_title('Elbow Method')

axes[1].plot(list(K_range), sil_scores, marker='o', color='darkorange')
axes[1].set_xlabel('Number of clusters (k)')
axes[1].set_ylabel('Silhouette Score')
axes[1].set_title('Silhouette Analysis')
plt.tight_layout()
plt.show()

best_k = list(K_range)[int(np.argmax(sil_scores))]
print(f"Best k by silhouette score: {best_k}")
```



```
Best k by silhouette score: 2
```

[ ]:
```
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df['KMeans_Cluster'] = kmeans.fit_predict(X_scaled)
df['KMeans_Cluster'].value_counts().sort_index()
```

[ ]:
```
KMeans_Cluster
0    209
1    113
Name: count, dtype: int64
```

[ ]:
```
cluster_profile = df.groupby('KMeans_Cluster')[cluster_features].mean()
cluster_profile
```

[ ]:
```
                      AtBat        Hits      HmRun       Runs        RBI  \
KMeans_Cluster
0                331.464115   85.373206   8.167464  42.961722  38.669856
1                472.415929  129.973451  15.584071  65.610619  65.336283

                     Walks      Years       CAtBat        CHits      CHmRun  \
KMeans_Cluster
0                31.574163   5.124402  1364.779904   353.937799   27.071770
1                52.000000  11.734513  5023.336283  1390.132743  147.946903

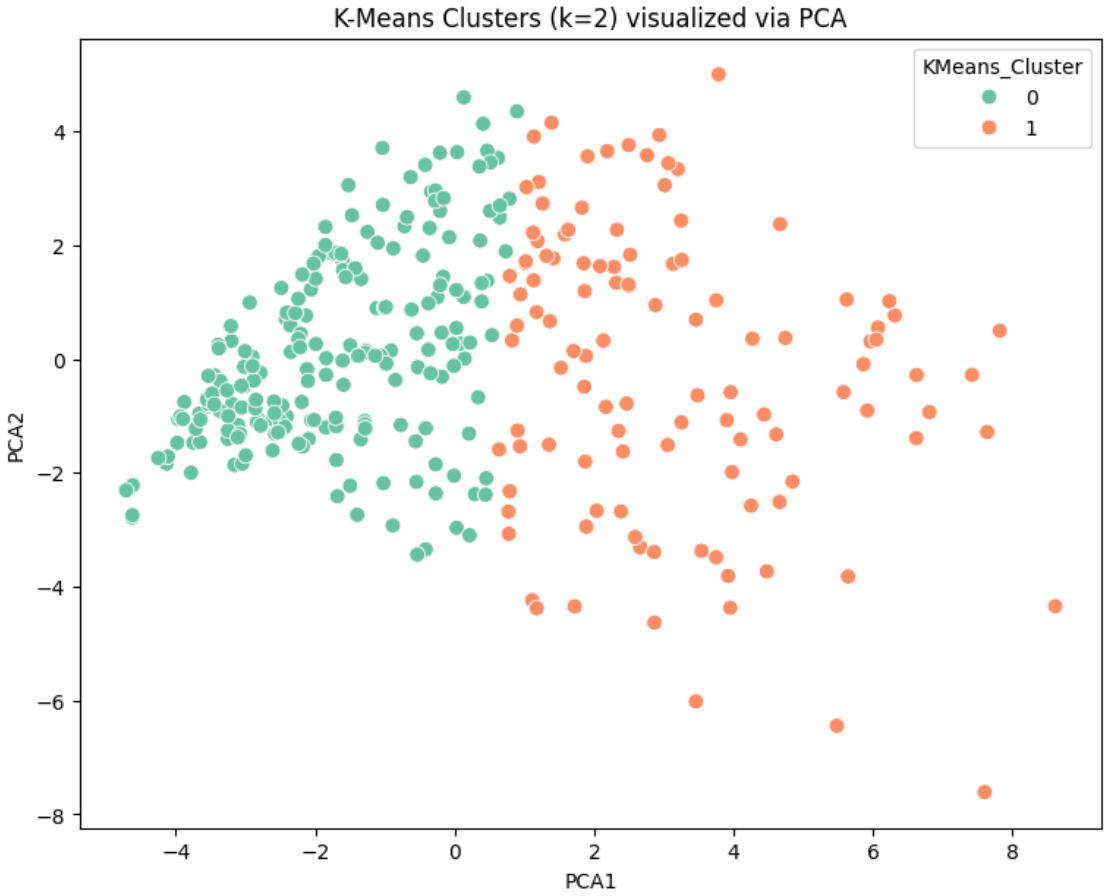
                      CRuns        CRBI      CWalks     PutOuts     Assists  \
KMeans_Cluster
0                172.033493  147.296651  121.019139  253.827751  102.315789
1                704.221239  668.256637  517.734513  353.876106  115.415929

                    Errors      Salary
KMeans_Cluster
0                8.086124  345.892349
1                7.955752  829.486779
```

[ ]:
```
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)
df['PCA1'], df['PCA2'] = X_pca[:, 0], X_pca[:, 1]

plt.figure(figsize=(9, 7))
sns.scatterplot(x='PCA1', y='PCA2', hue='KMeans_Cluster', data=df,␣
 ↪palette='Set2', s=60)
plt.title(f"K-Means Clusters (k={best_k}) visualized via PCA")
plt.show()

print(f"Explained variance ratio (PC1, PC2): {pca.explained_variance_ratio_}")
```




```
Explained variance ratio (PC1, PC2): [0.45456972 0.24566837]
```

[ ]:
```
plt.figure(figsize=(8, 5))
sns.boxplot(x='KMeans_Cluster', y='Salary', data=df)
plt.title("Salary distribution by K-Means cluster")
plt.show()
```



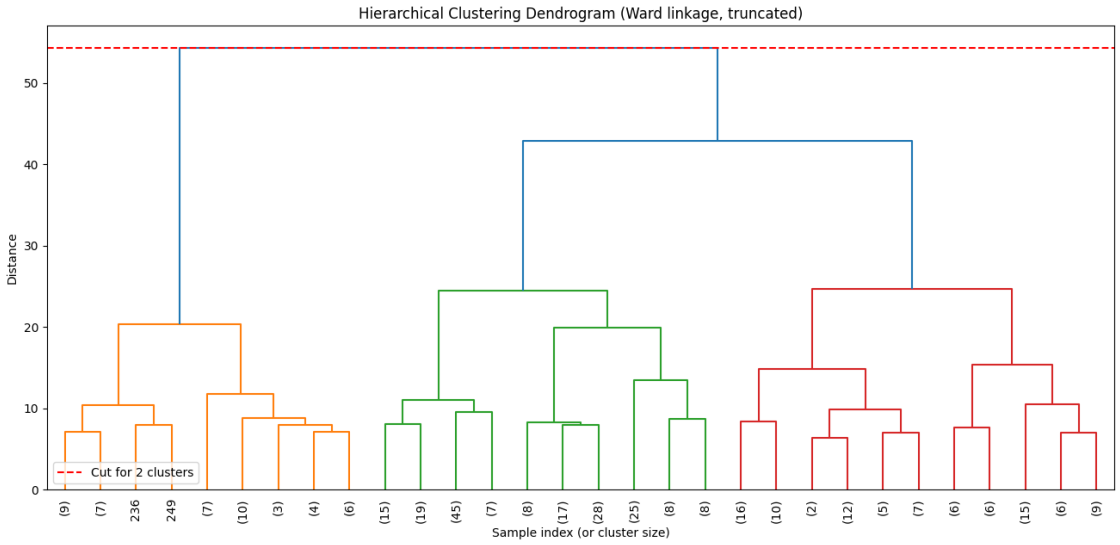
0.2.2 10. Hierarchical Clustering

Building a Ward-linkage dendrogram on the same standardized data, cutting at the same k, and comparing with K-Means results.

[ ]:
```
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from sklearn.cluster import AgglomerativeClustering

Z = linkage(X_scaled, method='ward')

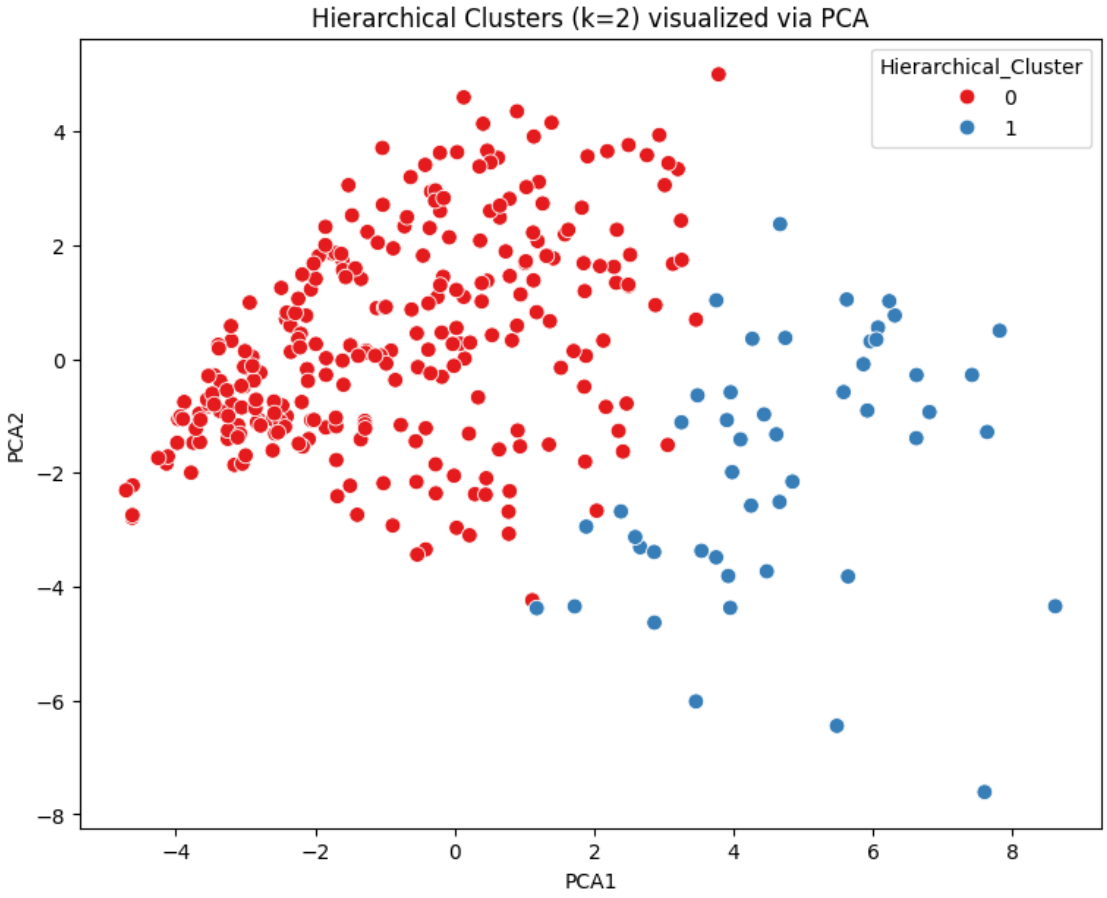
plt.figure(figsize=(16, 7))
dendrogram(Z, truncate_mode='lastp', p=30, leaf_rotation=90., leaf_font_size=10.
 ↪)
plt.title("Hierarchical Clustering Dendrogram (Ward linkage, truncated)")
plt.xlabel("Sample index (or cluster size)")
plt.ylabel("Distance")
plt.axhline(y=Z[-(best_k-1), 2], color='red', linestyle='--',
            label=f'Cut for {best_k} clusters')
plt.legend()
plt.show()
```



[ ]:
```
agg = AgglomerativeClustering(n_clusters=best_k, linkage='ward')
df['Hierarchical_Cluster'] = agg.fit_predict(X_scaled)
df['Hierarchical_Cluster'].value_counts().sort_index()
```

[ ]:
```
Hierarchical_Cluster
0    274
1     48
Name: count, dtype: int64
```

[ ]:
```
plt.figure(figsize=(9, 7))
sns.scatterplot(x='PCA1', y='PCA2', hue='Hierarchical_Cluster', data=df,␣
↪palette='Set1', s=60)
plt.title(f"Hierarchical Clusters (k={best_k}) visualized via PCA")
plt.show()
```



[ ]:
```
hier_profile = df.groupby('Hierarchical_Cluster')[cluster_features].mean()
hier_profile
```

[ ]:

Hierarchical_Cluster	AtBat	Hits	HmRun	Runs	RBI
0	372.711679	98.927007	9.711679	49.492701	45.003650
1	427.833333	113.000000	16.812500	59.000000	65.291667

Hierarchical_Cluster	Walks	Years	CAtBat	CHits
0	36.627737	6.051095	1908.660584	510.678832
1	50.812500	15.395833	6872.979167	1898.583333

Hierarchical_Cluster	CHmRun	CRuns	CRBI	CWalks
0	40.266423	249.635036	214.197080	174.744526
1	236.312500	981.916667	991.833333	748.270833

```
                          PutOuts     Assists    Errors      Salary
Hierarchical_Cluster
0                      288.667883  113.941606  8.456204  454.947602
1                      290.479167   66.791667  5.666667  861.830500
```

[ ]:
```
from sklearn.metrics import adjusted_rand_score, silhouette_score

ari = adjusted_rand_score(df['KMeans_Cluster'], df['Hierarchical_Cluster'])
sil_kmeans = silhouette_score(X_scaled, df['KMeans_Cluster'])
sil_hier = silhouette_score(X_scaled, df['Hierarchical_Cluster'])

print(f"Adjusted Rand Index (KMeans vs Hierarchical): {ari:.3f}")
print(f"Silhouette Score - K-Means:      {sil_kmeans:.3f}")
print(f"Silhouette Score - Hierarchical: {sil_hier:.3f}")

pd.crosstab(df['KMeans_Cluster'], df['Hierarchical_Cluster'],
            rownames=['KMeans'], colnames=['Hierarchical'])
```

```
Adjusted Rand Index (KMeans vs Hierarchical): 0.325
Silhouette Score - K-Means:      0.306
Silhouette Score - Hierarchical: 0.369
```

[ ]:
```
Hierarchical    0   1
KMeans
0             209   0
1              65  48
```

0.3 PHASE 3: Dimensionality reduction and model development

0.3.1 11. Setup for modelling

[ ]:
```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
from scipy import stats
import statsmodels.api as sm
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split, cross_val_score,␣
 ↪StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, confusion_matrix,␣
 ↪ConfusionMatrixDisplay,
                             classification_report, roc_curve, roc_auc_score)
from sklearn.decomposition import FactorAnalysis
from sklearn.manifold import MDS, Isomap, TSNE, trustworthiness
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import warnings; warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
```

```
url = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/Hitters.csv"
hit_raw = pd.read_csv(url)
print("Raw shape:", hit_raw.shape, "| missing Salary:", hit_raw['Salary'].
  ↪isna().sum())

df_full = hit_raw.copy()
df_full['Salary'] = df_full['Salary'].fillna(df_full['Salary'].median())

obs = hit_raw.dropna(subset=['Salary']).copy()
obs['Salary_log'] = np.log(obs['Salary'])
print("Rows with real salary:", obs.shape[0])
```

```
Raw shape: (322, 20) | missing Salary: 59
Rows with real salary: 263
```

Models use the 263 players with a recorded salary (no median-filled values). A fresh copy of the data (`hit_raw`) is loaded because `df` was modified in Sections 5 to 10.

0.3.2 12. Principal Component Analysis (PCA)

[ ]:
```
features = [c for c in hit_raw.columns if c not in␣
  ↪['Salary','League','Division','NewLeague']]
X = StandardScaler().fit_transform(obs[features])
pca = PCA().fit(X)
scores = pca.transform(X)

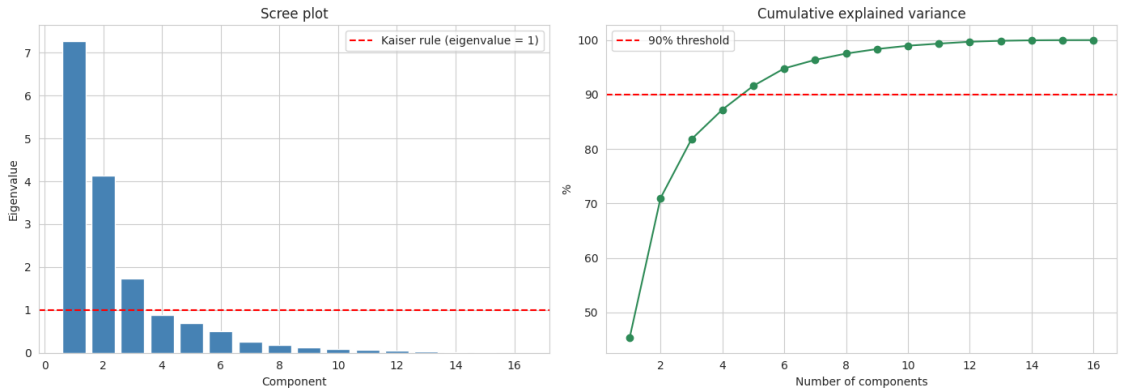
evr = pca.explained_variance_ratio_
cum = np.cumsum(evr)
pd.DataFrame({'PC':[f'PC{i+1}' for i in range(len(evr))],
              'Eigenvalue':pca.explained_variance_.round(3),
              'Variance %':(evr*100).round(2),
              'Cumulative %':(cum*100).round(2)}).head(8)
```

[ ]:

	PC	Eigenvalue	Variance %	Cumulative %
0	PC1	7.278	45.31	45.31
1	PC2	4.126	25.69	71.00
2	PC3	1.735	10.80	81.80
3	PC4	0.874	5.44	87.24
4	PC5	0.700	4.36	91.60
5	PC6	0.514	3.20	94.80
6	PC7	0.253	1.57	96.37
7	PC8	0.186	1.16	97.53

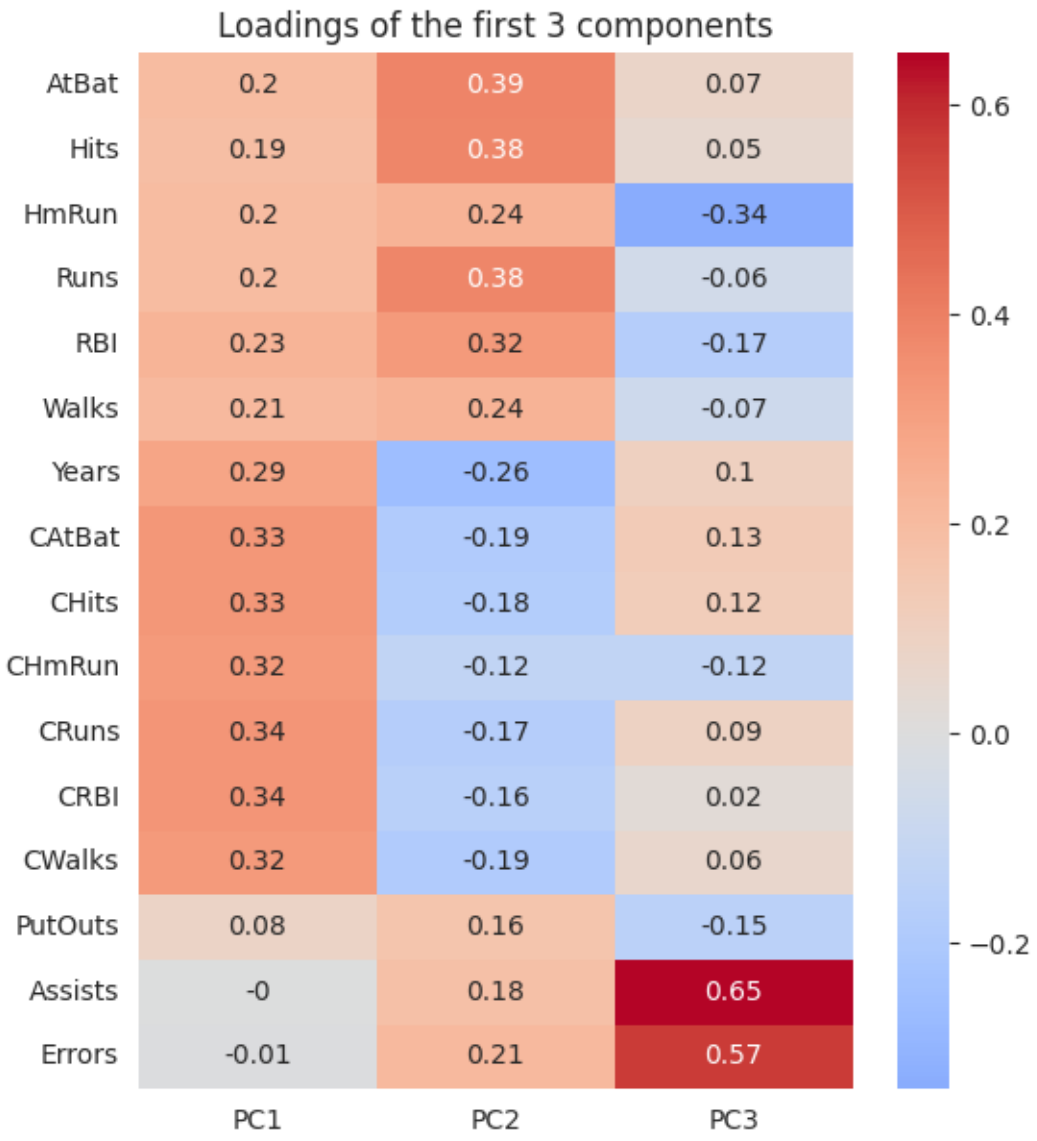
[ ]:
```
fig, ax = plt.subplots(1, 2, figsize=(14,5))
ax[0].bar(range(1,len(evr)+1), pca.explained_variance_, color='steelblue')
ax[0].axhline(1, color='red', ls='--', label='Kaiser rule (eigenvalue = 1)')
```

```
ax[0].set_title('Scree plot'); ax[0].set_xlabel('Component'); ax[0].
 ↪set_ylabel('Eigenvalue'); ax[0].legend()
ax[1].plot(range(1,len(cum)+1), cum*100, 'o-', color='seagreen')
ax[1].axhline(90, color='red', ls='--', label='90% threshold')
ax[1].set_title('Cumulative explained variance'); ax[1].set_xlabel('Number of␣
 ↪components'); ax[1].set_ylabel('%'); ax[1].legend()
plt.tight_layout(); plt.show()
print("Components needed for 80%:", np.argmax(cum>=0.80)+1, "| for 90%:", np.
 ↪argmax(cum>=0.90)+1)
print("Components with eigenvalue > 1:", (pca.explained_variance_>1).sum())
```




```
Components needed for 80%: 3 | for 90%: 5
Components with eigenvalue > 1: 3
```

[ ]:
```
loadings = pd.DataFrame(pca.components_[:3].T, index=features,␣
 ↪columns=['PC1','PC2','PC3']).round(2)
plt.figure(figsize=(6,7))
sns.heatmap(loadings, annot=True, cmap='coolwarm', center=0)
plt.title('Loadings of the first 3 components'); plt.show()
loadings
```



[ ]:

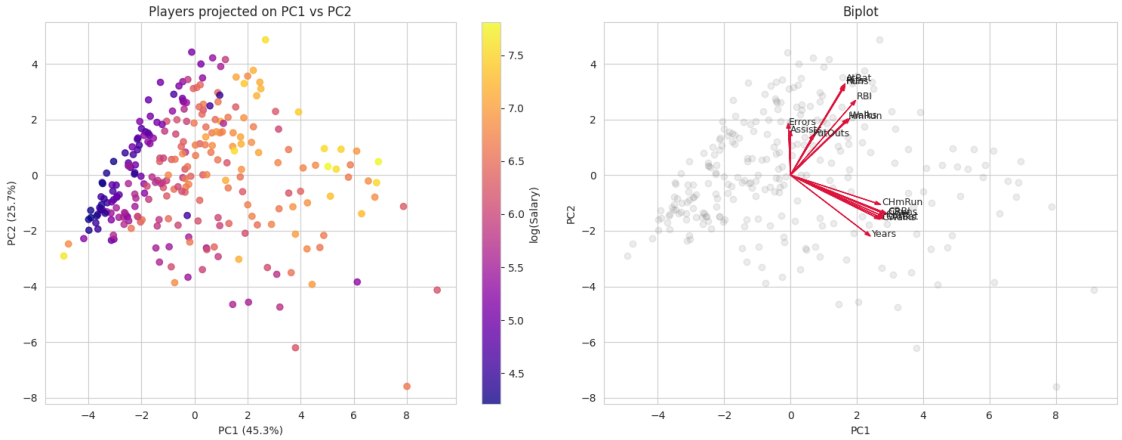
	PC1	PC2	PC3
AtBat	0.20	0.39	0.07
Hits	0.19	0.38	0.05
HmRun	0.20	0.24	-0.34
Runs	0.20	0.38	-0.06
RBI	0.23	0.32	-0.17
Walks	0.21	0.24	-0.07
Years	0.29	-0.26	0.10
CAtBat	0.33	-0.19	0.13
CHits	0.33	-0.18	0.12
CHmRun	0.32	-0.12	-0.12
CRuns	0.34	-0.17	0.09

```
CRBI      0.34 -0.16  0.02
CWalks    0.32 -0.19  0.06
PutOuts   0.08  0.16 -0.15
Assists -0.00  0.18  0.65
Errors  -0.01  0.21  0.57
```

PC1 = career experience, PC2 = current-season hitting, PC3 = fielding.

[ ]:
```
fig, ax = plt.subplots(1, 2, figsize=(15,6))
sc = ax[0].scatter(scores[:,0], scores[:,1], c=obs['Salary_log'],␣
 ↪cmap='plasma', alpha=0.8)
plt.colorbar(sc, ax=ax[0], label='log(Salary)')
ax[0].set_xlabel(f'PC1 ({evr[0]*100:.1f}%)'); ax[0].set_ylabel(f'PC2␣
 ↪({evr[1]*100:.1f}%)')
ax[0].set_title('Players projected on PC1 vs PC2')

ax[1].scatter(scores[:,0], scores[:,1], alpha=0.15, color='grey')
for i, f in enumerate(features):
    ax[1].arrow(0,0,pca.components_[0,i]*8, pca.components_[1,i]*8,␣
 ↪color='crimson', head_width=0.12)
    ax[1].text(pca.components_[0,i]*8.6, pca.components_[1,i]*8.6, f,␣
 ↪fontsize=9)
ax[1].set_xlabel('PC1'); ax[1].set_ylabel('PC2'); ax[1].set_title('Biplot')
plt.tight_layout(); plt.show()
```

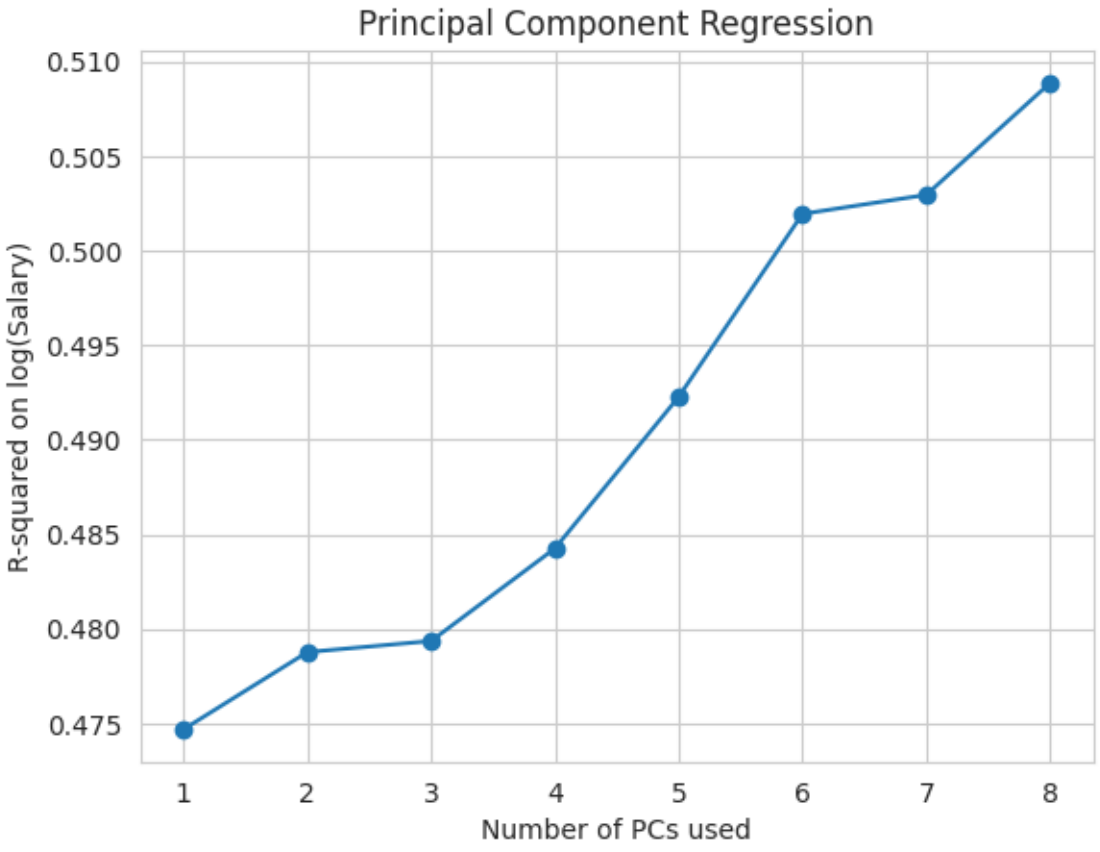


PC1 correlates about 0.69 with log-salary; principal component regression:

[ ]:
```
pc_corr = pd.Series([np.corrcoef(scores[:,i], obs['Salary_log'])[0,1] for i in␣
 ↪range(6)],
                    index=[f'PC{i+1}' for i in range(6)]).round(3)
print(pc_corr)
```

```
r2 = []
for k in range(1,9):
    m = sm.OLS(obs['Salary_log'].values, sm.add_constant(scores[:,:k])).fit()
    r2.append(m.rsquared)
plt.plot(range(1,9), r2, 'o-'); plt.xlabel('Number of PCs used'); plt.
 ↪ylabel('R-squared on log(Salary)')
plt.title('Principal Component Regression'); plt.show()
print(np.round(r2,3))
```

```
PC1    0.689
PC2    0.064
PC3    0.024
PC4    0.070
PC5    0.090
PC6   -0.098
dtype: float64
```




```
[0.475 0.479 0.479 0.484 0.492 0.502 0.503 0.509]
```

0.3.3 13. Singular Value Decomposition (SVD)

X = U Σ V , and the PCA eigenvalues equal ² / (n-1).

[ ]:
```
U, S, Vt = np.linalg.svd(X, full_matrices=False)
n_obs = X.shape[0]

check = pd.DataFrame({'Singular value': S[:5],
                      'sigma^2/(n-1)': S[:5]**2 / (n_obs - 1),
                      'PCA eigenvalue': pca.explained_variance_[:5]},␣
 ↪index=[f'PC{i+1}' for i in range(5)]).round(4)
same_scores = np.allclose(np.abs(U * S), np.abs(scores))
print("PCA scores == U*Sigma (up to sign):", same_scores)
check
```

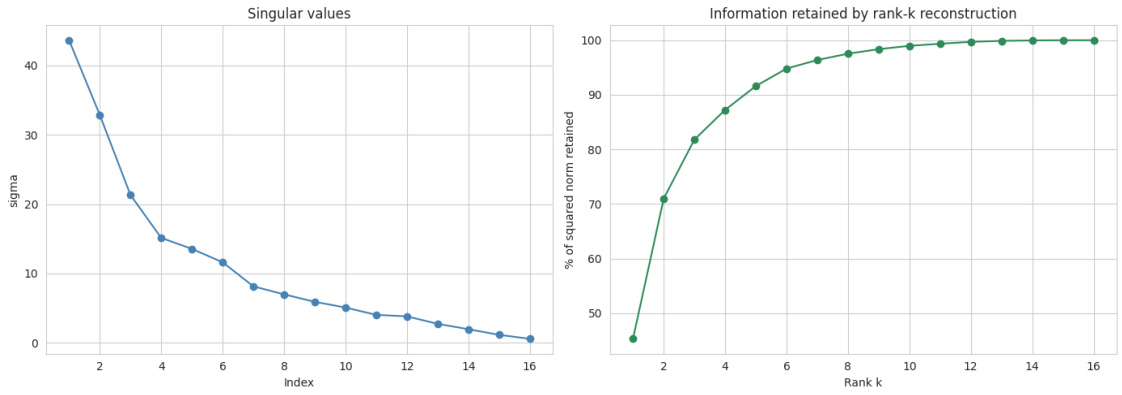
```
PCA scores == U*Sigma (up to sign): True
```

[ ]:

	Singular value	sigma^2/(n-1)	PCA eigenvalue
PC1	43.6661	7.2776	7.2776
PC2	32.8769	4.1255	4.1255
PC3	21.3180	1.7346	1.7346
PC4	15.1298	0.8737	0.8737
PC5	13.5428	0.7000	0.7000

[ ]:
```
ks = np.arange(1, len(S) + 1)
rel_err = np.array([np.linalg.norm(X - (U[:, :k] * S[:k]) @ Vt[:k])**2 / np.
 ↪linalg.norm(X)**2 for k in ks])

fig, ax = plt.subplots(1, 2, figsize=(14, 5))
ax[0].plot(ks, S, 'o-', color='steelblue'); ax[0].set_title('Singular values');␣
 ↪ax[0].set_xlabel('Index'); ax[0].set_ylabel('sigma')
ax[1].plot(ks, (1 - rel_err) * 100, 'o-', color='seagreen')
ax[1].set_title('Information retained by rank-k reconstruction'); ax[1].
 ↪set_xlabel('Rank k'); ax[1].set_ylabel('% of squared norm retained')
plt.tight_layout(); plt.show()
pd.DataFrame({'k': [1, 2, 3, 5, 8], 'Relative reconstruction error':␣
 ↪rel_err[[0, 1, 2, 4, 7]].round(3),
              'Retained %': ((1 - rel_err[[0, 1, 2, 4, 7]]) * 100).round(1)})
```



[ ]:

	k	Relative reconstruction error	Retained %
0	1	0.547	45.3
1	2	0.290	71.0
2	3	0.182	81.8
3	5	0.084	91.6
4	8	0.025	97.5

SVD matches PCA exactly; a rank-3 reconstruction keeps about 82% of the information.

0.3.4 14. Factor Analysis and Intrinsic Dimensionality

[ ]:
```
p_vars = X.shape[1]
R = np.corrcoef(X, rowvar=False)
chi2_stat = -(n_obs - 1 - (2 * p_vars + 5) / 6) * np.log(np.linalg.det(R))
dof = p_vars * (p_vars - 1) / 2
p_bart = stats.chi2.sf(chi2_stat, dof)
print(f"Bartlett chi-square = {chi2_stat:.1f}, dof = {int(dof)}, p-value = {'<␣
 ↪1e-300' if p_bart == 0 else format(p_bart, '.3g')}")
```

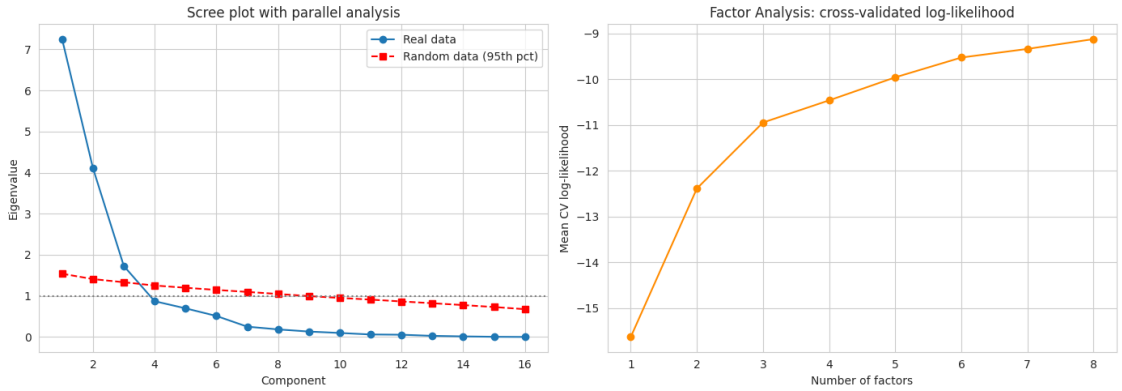
```
Bartlett chi-square = 7720.8, dof = 120, p-value = < 1e-300
```

[ ]:
```
eig_real = np.sort(np.linalg.eigvalsh(R))[::-1]

kaiser = int((eig_real > 1).sum())
var80 = int(np.argmax(np.cumsum(eig_real) / eig_real.sum() >= 0.80) + 1)
rng = np.random.default_rng(0)
sim = np.array([np.sort(np.linalg.eigvalsh(np.corrcoef(rng.
 ↪standard_normal((n_obs, p_vars)), rowvar=False)))[::-1]
                for _ in range(300)])
thr = np.percentile(sim, 95, axis=0)
parallel = int((eig_real > thr).sum())
fa_ll = [cross_val_score(FactorAnalysis(n_components=k, random_state=0), X,␣
 ↪cv=5).mean() for k in range(1, 9)]
```

```
fig, ax = plt.subplots(1, 2, figsize=(14, 5))
ax[0].plot(range(1, p_vars + 1), eig_real, 'o-', label='Real data')
ax[0].plot(range(1, p_vars + 1), thr, 's--', color='red', label='Random data␣
 ↪(95th pct)')
ax[0].axhline(1, color='grey', ls=':'); ax[0].set_title('Scree plot with␣
 ↪parallel analysis')
ax[0].set_xlabel('Component'); ax[0].set_ylabel('Eigenvalue'); ax[0].legend()
ax[1].plot(range(1, 9), fa_ll, 'o-', color='darkorange'); ax[1].
 ↪set_title('Factor Analysis: cross-validated log-likelihood')
ax[1].set_xlabel('Number of factors'); ax[1].set_ylabel('Mean CV␣
 ↪log-likelihood')
plt.tight_layout(); plt.show()

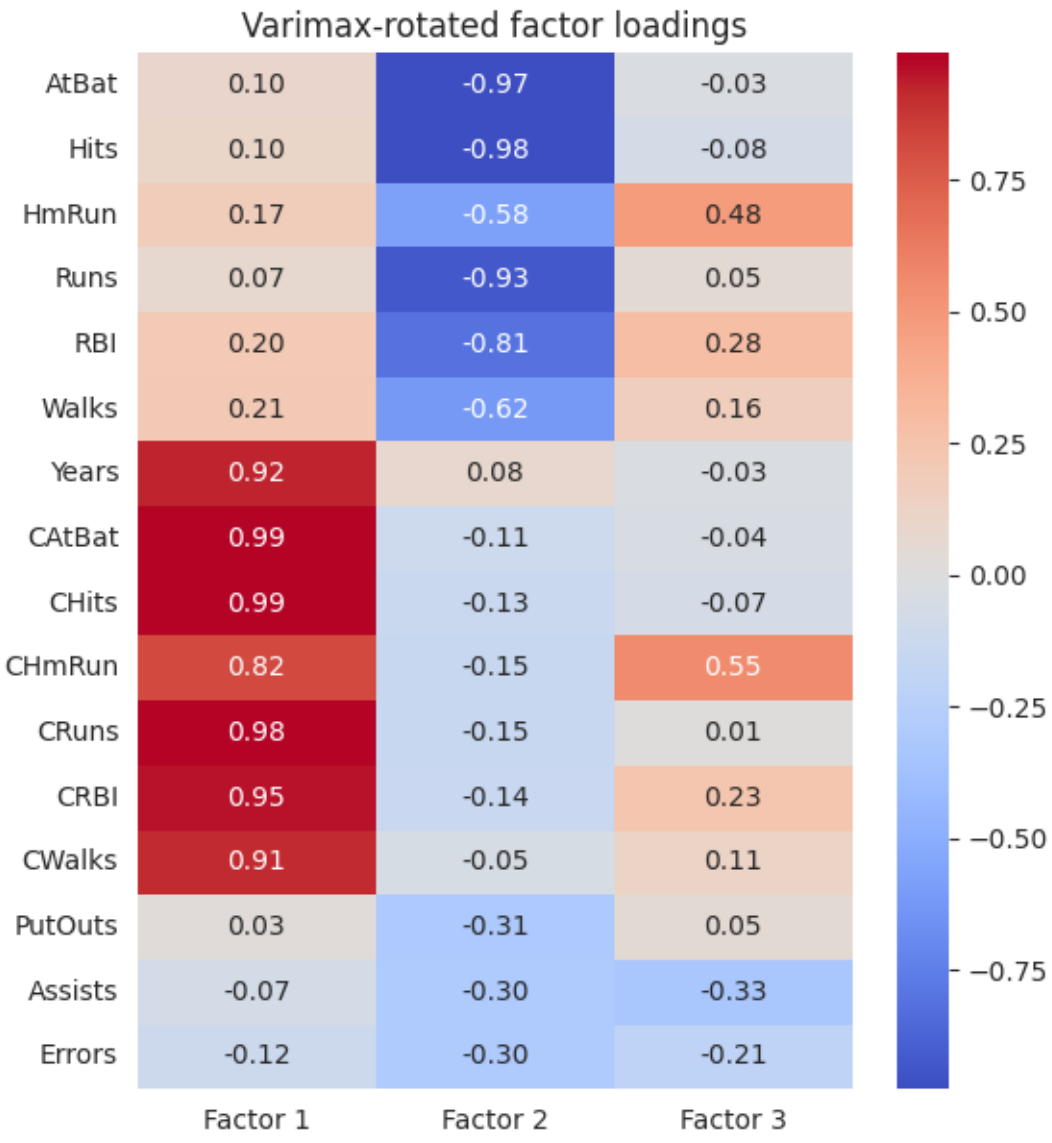
pd.Series({'Kaiser (eigenvalue > 1)': kaiser, '80% variance': var80, 'Parallel␣
 ↪analysis': parallel}, name='Suggested dimensions')
```



[ ]:
```
Kaiser (eigenvalue > 1)    3
80% variance               3
Parallel analysis          3
Name: Suggested dimensions, dtype: int64
```

[ ]:
```
fa3 = FactorAnalysis(n_components=3, rotation='varimax', random_state=0).fit(X)
fa_load = pd.DataFrame(fa3.components_.T, index=features, columns=['Factor 1',␣
 ↪'Factor 2', 'Factor 3'])
uniq = pd.Series(fa3.noise_variance_, index=features, name='Uniqueness')

plt.figure(figsize=(6, 7))
sns.heatmap(fa_load, annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Varimax-rotated factor loadings'); plt.show()
pd.concat([fa_load.round(2), uniq.round(2)], axis=1)
```



[ ]:

	Factor 1	Factor 2	Factor 3	Uniqueness
AtBat	0.10	-0.97	-0.03	0.04
Hits	0.10	-0.98	-0.08	0.03
HmRun	0.17	-0.58	0.48	0.40
Runs	0.07	-0.93	0.05	0.13
RBI	0.20	-0.81	0.28	0.22
Walks	0.21	-0.62	0.16	0.55
Years	0.92	0.08	-0.03	0.14
CAtBat	0.99	-0.11	-0.04	0.01
CHits	0.99	-0.13	-0.07	0.00
CHmRun	0.82	-0.15	0.55	0.01
CRuns	0.98	-0.15	0.01	0.02

```
CRBI         0.95     -0.14      0.23        0.02
CWalks       0.91     -0.05      0.11        0.16
PutOuts      0.03     -0.31      0.05        0.90
Assists     -0.07     -0.30     -0.33        0.80
Errors      -0.12     -0.30     -0.21        0.85
```

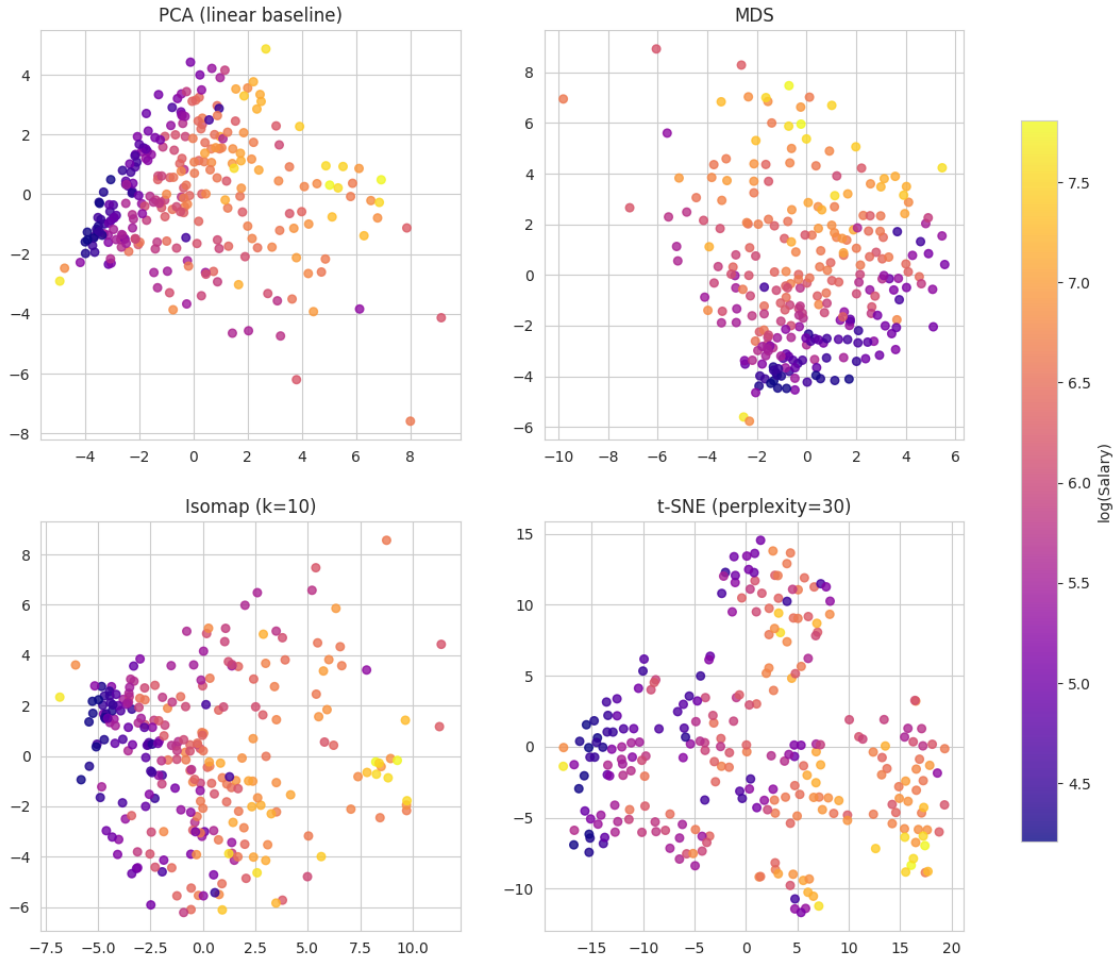
Kaiser, 80% variance and parallel analysis all give 3 dimensions. Factors: career volume, current-season hitting, power hitting (signs are arbitrary).

0.3.5 15. Non-linear methods: MDS, Isomap, t-SNE and Self-Organizing Map

[ ]:
```
embeddings = {
    'PCA (linear baseline)': PCA(n_components=2).fit_transform(X),
    'MDS': MDS(n_components=2, n_init=4, random_state=0).fit_transform(X),
    'Isomap (k=10)': Isomap(n_neighbors=10, n_components=2).fit_transform(X),
    't-SNE (perplexity=30)': TSNE(n_components=2, perplexity=30, init='pca',␣
 ↪random_state=0).fit_transform(X),
}

fig, axes = plt.subplots(2, 2, figsize=(14, 11))
for ax, (name, Z) in zip(axes.ravel(), embeddings.items()):
    sc_ = ax.scatter(Z[:, 0], Z[:, 1], c=obs['Salary_log'], cmap='plasma',␣
 ↪alpha=0.8, s=30)
    ax.set_title(name)
fig.colorbar(sc_, ax=axes, label='log(Salary)', shrink=0.8)
plt.show()

trust = pd.Series({name: trustworthiness(X, Z, n_neighbors=10) for name, Z in␣
 ↪embeddings.items()}, name='Trustworthiness').round(3)
trust
```



[ ]:
```
PCA (linear baseline)    0.909
MDS                      0.923
Isomap (k=10)            0.897
t-SNE (perplexity=30)    0.971
Name: Trustworthiness, dtype: float64
```

Trustworthiness is about 0.90 or higher for all methods (t-SNE best at 0.97); PCA is nearly as good, so there is little non-linear structure.

[ ]:
```
def train_som(data, grid_x=8, grid_y=8, iters=8000, lr0=0.5, seed=0):
    rng = np.random.default_rng(seed)
    W = rng.standard_normal((grid_x, grid_y, data.shape[1])) * 0.1
    gx, gy = np.meshgrid(np.arange(grid_x), np.arange(grid_y), indexing='ij')
    sigma0 = max(grid_x, grid_y) / 2
    for t in range(iters):
        x = data[rng.integers(len(data))]
        d = ((W - x) ** 2).sum(axis=2)
```

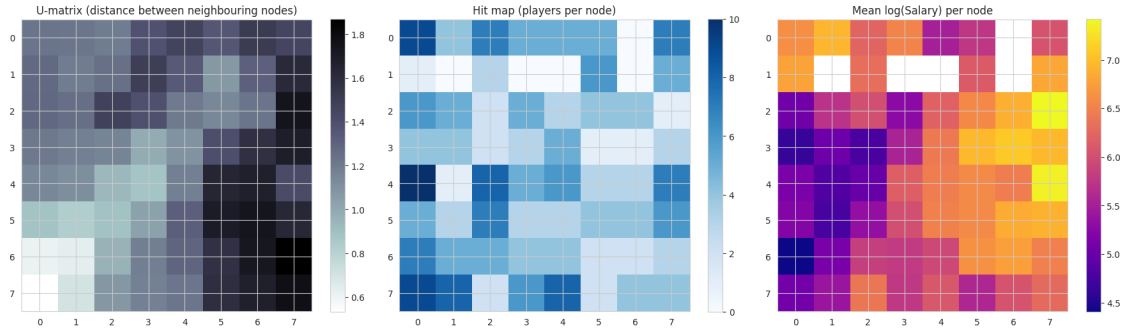
```
        bi, bj = np.unravel_index(d.argmin(), d.shape)
        lr = lr0 * np.exp(-3 * t / iters)
        sigma = sigma0 * np.exp(-3 * t / iters) + 0.5
        h = np.exp(-((gx - bi) ** 2 + (gy - bj) ** 2) / (2 * sigma ** 2))
        W += lr * h[..., None] * (x - W)
    return W

som = train_som(X)
gx_, gy_ = som.shape[:2]

bmus = np.array([np.unravel_index(((som - x) ** 2).sum(axis=2).argmin(), (gx_,␣
 ↪gy_)) for x in X])
q_err = np.mean([np.sqrt(((som - x) ** 2).sum(axis=2).min()) for x in X])
node_sal = np.full((gx_, gy_), np.nan); node_cnt = np.zeros((gx_, gy_),␣
 ↪dtype=int)
for i in range(gx_):
    for j in range(gy_):
        m = (bmus[:, 0] == i) & (bmus[:, 1] == j); node_cnt[i, j] = m.sum()
        if m.any(): node_sal[i, j] = obs['Salary_log'].values[m].mean()

umat = np.zeros((gx_, gy_))
for i in range(gx_):
    for j in range(gy_):
        nb_d = [np.linalg.norm(som[i, j] - som[a, b]) for a, b in [(i-1, j),␣
 ↪(i+1, j), (i, j-1), (i, j+1)]
                if 0 <= a < gx_ and 0 <= b < gy_]
        umat[i, j] = np.mean(nb_d)

fig, ax = plt.subplots(1, 3, figsize=(18, 5))
im0 = ax[0].imshow(umat, cmap='bone_r'); ax[0].set_title('U-matrix (distance␣
 ↪between neighbouring nodes)'); plt.colorbar(im0, ax=ax[0])
im1 = ax[1].imshow(node_cnt, cmap='Blues'); ax[1].set_title('Hit map (players␣
 ↪per node)'); plt.colorbar(im1, ax=ax[1])
im2 = ax[2].imshow(node_sal, cmap='plasma'); ax[2].set_title('Mean log(Salary)␣
 ↪per node'); plt.colorbar(im2, ax=ax[2])
plt.tight_layout(); plt.show()
print(f"Quantization error: {q_err:.3f} | occupied nodes: {(node_cnt > 0).
 ↪sum()} of {gx_ * gy_}")
```




```
Quantization error: 1.631 | occupied nodes: 59 of 64
```

The SOM salary map shows a low-to-high gradient although salary was not used in training.

0.3.6 16. Simple Linear Regression (SLR)

[ ]:
```
cands = features
res = {c: sm.OLS(obs['Salary_log'], sm.add_constant(obs[c])).fit().rsquared for␣
 ↪c in cands}
pd.Series(res).sort_values(ascending=False).round(3).head(8)
```

[ ]:
```
CRuns     0.386
CHits     0.384
CAtBat    0.373
CRBI      0.363
CWalks    0.299
Years     0.289
CHmRun    0.275
Hits      0.202
dtype: float64
```

[ ]:
```
predictor = 'CRuns'
X1 = sm.add_constant(obs[predictor]); y = obs['Salary_log']
slr = sm.OLS(y, X1).fit()
print(slr.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:             Salary_log   R-squared:                       0.386
Model:                            OLS   Adj. R-squared:                  0.383
Method:                 Least Squares   F-statistic:                     163.9
Date:                Thu, 15 Oct 2026   Prob (F-statistic):           1.89e-29
Time:                        19:42:46   Log-Likelihood:                -277.71
No. Observations:                 263   AIC:                             559.4
Df Residuals:                     261   BIC:                             566.6
Df Model:                           1
```

```
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          5.3249      0.064     83.496      0.000       5.199       5.450
CRuns          0.0017      0.000     12.803      0.000       0.001       0.002
==============================================================================
Omnibus:                        0.529   Durbin-Watson:                   1.962
Prob(Omnibus):                  0.768   Jarque-Bera (JB):                0.430
Skew:                          -0.098   Prob(JB):                        0.807
Kurtosis:                       3.022   Cond. No.                         725.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

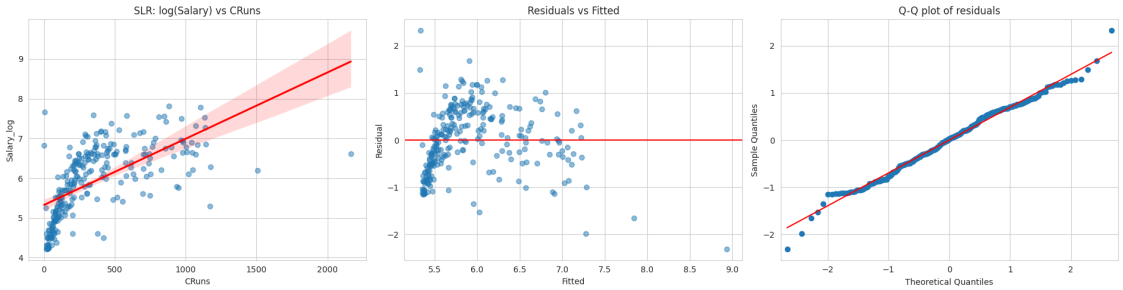
Significant positive slope; R-squared about 0.39.

[ ]:
```
b0, b1 = slr.params
print(f"log(Salary) = {b0:.4f} + {b1:.5f} x {predictor}")
print(f"Each extra 100 career runs -> salary x {np.exp(100*b1):.2f} (about␣
 ↪{100*(np.exp(100*b1)-1):.0f}% higher)")

fig, ax = plt.subplots(1, 3, figsize=(19,5))
sns.regplot(x=obs[predictor], y=y, ax=ax[0], scatter_kws={'alpha':0.5},␣
 ↪line_kws={'color':'red'})
ax[0].set_title(f'SLR: log(Salary) vs {predictor}')
fitted, resid = slr.fittedvalues, slr.resid
ax[1].scatter(fitted, resid, alpha=0.5); ax[1].axhline(0, color='red')
ax[1].set_xlabel('Fitted'); ax[1].set_ylabel('Residual'); ax[1].
 ↪set_title('Residuals vs Fitted')
sm.qqplot(resid, line='s', ax=ax[2]); ax[2].set_title('Q-Q plot of residuals')
plt.tight_layout(); plt.show()

print("Shapiro-Wilk on residuals: p =", round(stats.shapiro(resid)[1],4))
from statsmodels.stats.diagnostic import het_breuschpagan
print("Breusch-Pagan (equal variance) p =", round(het_breuschpagan(resid,␣
 ↪X1)[1],4))
```

```
log(Salary) = 5.3249 + 0.00167 x CRuns
Each extra 100 career runs -> salary x 1.18 (about 18% higher)
```




```
Shapiro-Wilk on residuals: p = 0.1566
Breusch-Pagan (equal variance) p = 0.017
```

Residuals look normal (Shapiro p about 0.16), but Breusch-Pagan (p about 0.017) shows mild unequal variance.

[ ]:
```
Xtr, Xte, ytr, yte = train_test_split(obs[[predictor]], y, test_size=0.25,␣
↪random_state=42)
m = sm.OLS(ytr, sm.add_constant(Xtr)).fit()
pred = m.predict(sm.add_constant(Xte))
rmse = np.sqrt(np.mean((yte-pred)**2))
r2_te = 1 - ((yte-pred)**2).sum()/((yte-yte.mean())**2).sum()
print(f"Test R^2 = {r2_te:.3f} | Test RMSE (log scale) = {rmse:.3f}")

new = pd.DataFrame({'const':1,'CRuns':[100,400,800]})
pr = slr.get_prediction(new).summary_frame(alpha=0.05)
out = np.exp(pr[['mean','obs_ci_lower','obs_ci_upper']])
out.index = ['100 runs','400 runs','800 runs']; out.columns=['Pred salary␣
↪(K$)','95% PI low','95% PI high']
out.round(0)
```

```
Test R^2 = 0.380 | Test RMSE (log scale) = 0.663
```

[ ]:

	Pred salary (K$)	95% PI low	95% PI high
100 runs	243.0	61.0	964.0
400 runs	400.0	101.0	1587.0
800 runs	780.0	196.0	3106.0

16.1 Evaluation and accuracy

[ ]:
```
Xtr_s, Xte_s, ytr_s, yte_s = train_test_split(obs[[predictor]],␣
↪obs['Salary_log'], test_size=0.25, random_state=42)
from sklearn.linear_model import LinearRegression
lin = LinearRegression().fit(Xtr_s, ytr_s)
baseline = np.full(len(yte_s), ytr_s.mean())

def report(y_true, y_pred, n_pred=1):
    n_ = len(y_true); r2_ = r2_score(y_true, y_pred)
```

```
    sal_true, sal_pred = np.exp(y_true), np.exp(y_pred)
    return {'R2': r2_, 'Adj R2': 1 - (1 - r2_) * (n_ - 1) / (n_ - n_pred - 1),
            'RMSE (log scale)': np.sqrt(mean_squared_error(y_true, y_pred)),
            'MAE (log scale)': mean_absolute_error(y_true, y_pred),
            'MAE ($K)': mean_absolute_error(sal_true, sal_pred),
            'Within 50% of actual salary': np.mean(np.abs(sal_pred - sal_true) /
 ↪ sal_true <= 0.5)}

metrics = pd.DataFrame({'Train': report(ytr_s, lin.predict(Xtr_s)),
                        'Test': report(yte_s, lin.predict(Xte_s)),
                        'Test: baseline (mean)': report(yte_s, baseline,␣
 ↪n_pred=0)}).round(3)

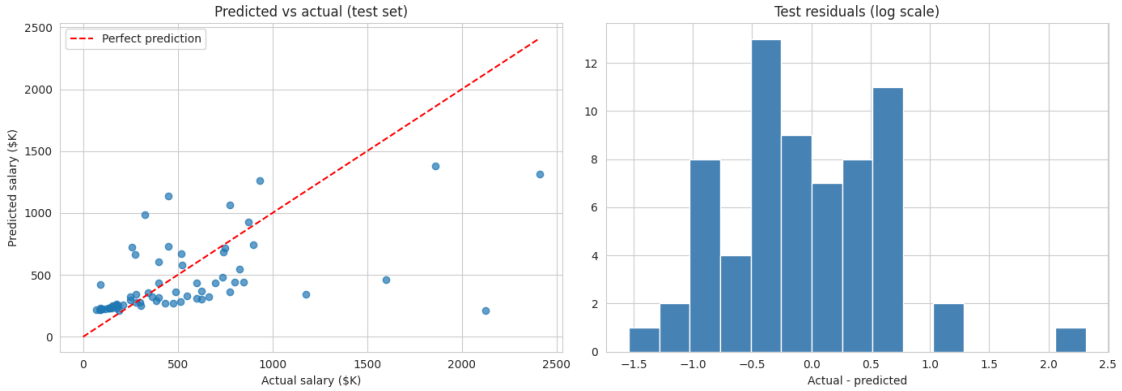
from sklearn.model_selection import KFold
cv_r2 = cross_val_score(LinearRegression(), obs[[predictor]], obs['Salary_log'],
                        cv=KFold(5, shuffle=True, random_state=42),␣
 ↪scoring='r2')
print("5-fold CV R2:", cv_r2.round(3), "| mean = %.3f, std = %.3f" % (cv_r2.
 ↪mean(), cv_r2.std()))
metrics
```

```
5-fold CV R2: [0.354 0.385 0.412 0.346 0.226] | mean = 0.345, std = 0.064
```

[ ]:

	Train	Test	Test: baseline (mean)
R2	0.387	0.380	-0.002
Adj R2	0.384	0.371	-0.002
RMSE (log scale)	0.706	0.663	0.843
MAE (log scale)	0.580	0.527	0.697
MAE ($K)	310.273	246.791	310.154
Within 50% of actual salary	0.533	0.591	0.470

[ ]:
```
pred_te = np.exp(lin.predict(Xte_s)); act_te = np.exp(yte_s)
fig, ax = plt.subplots(1, 2, figsize=(14, 5))
ax[0].scatter(act_te, pred_te, alpha=0.7); lim = [0, max(act_te.max(), pred_te.
 ↪max())]
ax[0].plot(lim, lim, 'r--', label='Perfect prediction'); ax[0].
 ↪set_xlabel('Actual salary ($K)'); ax[0].set_ylabel('Predicted salary ($K)')
ax[0].set_title('Predicted vs actual (test set)'); ax[0].legend()
ax[1].hist(yte_s - lin.predict(Xte_s), bins=15, color='steelblue',␣
 ↪edgecolor='white')
ax[1].set_title('Test residuals (log scale)'); ax[1].set_xlabel('Actual -␣
 ↪predicted')
plt.tight_layout(); plt.show()
```



- Train (0.39), test (0.38) and CV (about 0.35) R-squared are close, so no over-fitting; the mean baseline has R-squared of about 0.
- Typical error is a factor of about 1.7 to 1.9 (test RMSE 0.66 vs 0.84 for the baseline).
- The model explains the trend but not individual salaries.

0.3.7 17. Classification: Logistic Regression

Target: high vs low salary (above the median).

[ ]:
```
med = obs['Salary'].median()
obs['HighSalary'] = (obs['Salary'] > med).astype(int)
print("Median salary (K$):", med)
print(obs['HighSalary'].value_counts())

cls_feats = features + ['League','Division']
Xc = obs[cls_feats].copy()
Xc['League'] = (Xc['League']=='N').astype(int)
Xc['Division'] = (Xc['Division']=='W').astype(int)
yc = obs['HighSalary']

Xtr, Xte, ytr, yte = train_test_split(Xc, yc, test_size=0.25, stratify=yc,␣
 ↪random_state=42)
sc = StandardScaler().fit(Xtr)
clf = LogisticRegression(max_iter=1000)
clf.fit(sc.transform(Xtr), ytr)
yp = clf.predict(sc.transform(Xte)); yprob = clf.predict_proba(sc.
 ↪transform(Xte))[:,1]
print(f"Test accuracy: {accuracy_score(yte,yp):.3f} | ROC-AUC:␣
 ↪{roc_auc_score(yte,yprob):.3f}")
print(classification_report(yte, yp, target_names=['Low','High']))
```

```
Median salary (K$): 425.0
HighSalary
0    134
```

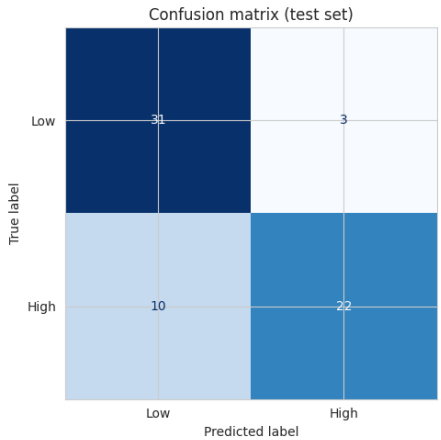
```
1    129
Name: count, dtype: int64
Test accuracy: 0.803 | ROC-AUC: 0.877
              precision    recall  f1-score   support

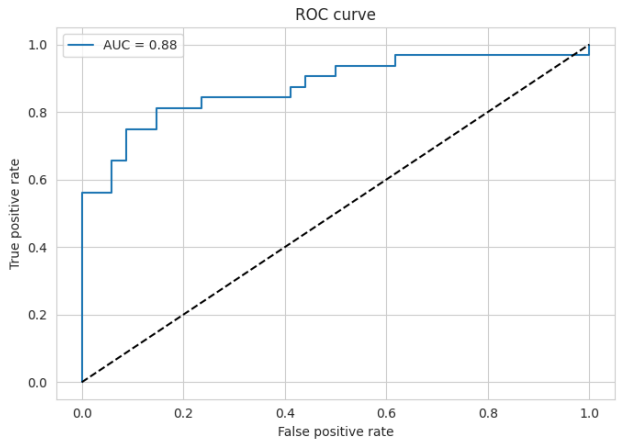
         Low       0.76      0.91      0.83        34
        High       0.88      0.69      0.77        32

    accuracy                           0.80        66
   macro avg       0.82      0.80      0.80        66
weighted avg       0.82      0.80      0.80        66
```

[ ]:
```
fig, ax = plt.subplots(1, 2, figsize=(13,5))
ConfusionMatrixDisplay(confusion_matrix(yte,yp), display_labels=['Low','High']).
 ↪plot(ax=ax[0], cmap='Blues', colorbar=False)
ax[0].set_title('Confusion matrix (test set)')
fpr, tpr, _ = roc_curve(yte, yprob)
ax[1].plot(fpr, tpr, label=f'AUC = {roc_auc_score(yte,yprob):.2f}'); ax[1].
 ↪plot([0,1],[0,1],'k--')
ax[1].set_xlabel('False positive rate'); ax[1].set_ylabel('True positive rate');
 ↪ ax[1].set_title('ROC curve'); ax[1].legend()
plt.tight_layout(); plt.show()

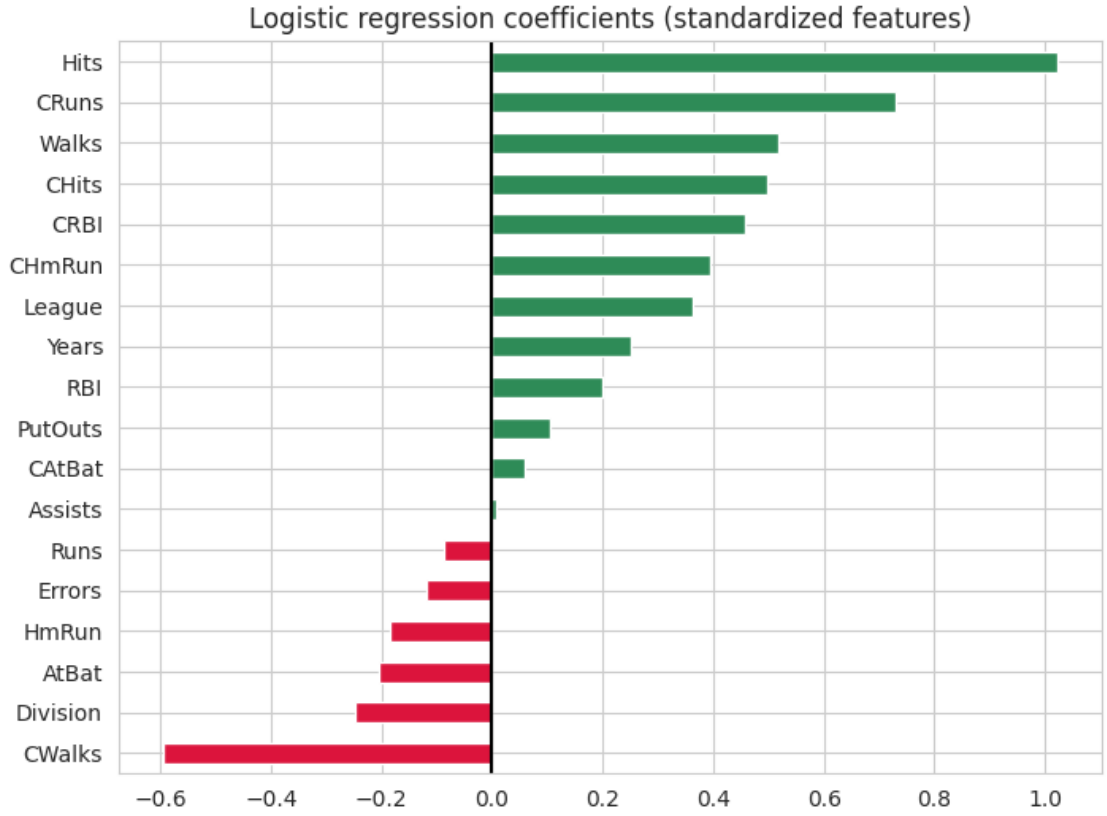
cv = cross_val_score(LogisticRegression(max_iter=1000),
                     StandardScaler().fit_transform(Xc), yc,
                     cv=StratifiedKFold(5, shuffle=True, random_state=42),␣
 ↪scoring='accuracy')
print("5-fold CV accuracy:", cv.round(3), "| mean =", cv.mean().round(3))
```






```
5-fold CV accuracy: [0.774 0.83  0.774 0.904 0.808] | mean = 0.818
```

[ ]:
```
coefs = pd.Series(clf.coef_[0], index=cls_feats).sort_values()
plt.figure(figsize=(8,6))
coefs.plot(kind='barh', color=np.where(coefs>0,'seagreen','crimson'))
plt.title('Logistic regression coefficients (standardized features)'); plt.
  ↪axvline(0,color='k'); plt.show()
coefs.round(2).sort_values(key=abs, ascending=False).head(6)
```



[ ]:
```
Hits      1.02
CRuns     0.73
CWalks   -0.59
Walks     0.52
CHits     0.50
CRBI      0.46
dtype: float64
```

Test accuracy about 80%, AUC about 0.88, CV about 0.82. Five PCs give nearly the same accuracy.

[ ]:
```
pca_c = PCA(n_components=5).fit(sc.transform(Xtr))
Ztr, Zte = pca_c.transform(sc.transform(Xtr)), pca_c.transform(sc.
  ↪transform(Xte))
clf_pc = LogisticRegression(max_iter=1000).fit(Ztr, ytr)
```

```
print(f"Logistic on 5 PCs  -> accuracy {accuracy_score(yte, clf_pc.
  ↪predict(Zte)):.3f}, AUC {roc_auc_score(yte, clf_pc.predict_proba(Zte)[:,1]):.
  ↪3f}")
print(f"Logistic on all {len(cls_feats)} features -> accuracy␣
  ↪{accuracy_score(yte,yp):.3f}, AUC {roc_auc_score(yte,yprob):.3f}")
```

```
Logistic on 5 PCs  -> accuracy 0.773, AUC 0.871
Logistic on all 18 features -> accuracy 0.803, AUC 0.877
```

0.3.8 18. Overall Summary

- **Phase 1:** loaded, cleaned (median-imputed Salary, encoded categoricals, log Salary) and explored the data with 1D, 2D and 3D analyses.
- **Phase 2:** K-Means and Ward clustering, k = 2 (ARI 0.325; silhouette 0.306 and 0.369).
- **PCA / SVD / FA:** the 16 statistics reduce to 3 dimensions (about 82% of variance): career experience, current-season hitting, and power or fielding.
- **MDS, Isomap, t-SNE, SOM:** all show a smooth salary gradient; PCA is nearly as good as the non-linear methods.
- **SLR:** log(Salary) on CRuns, R-squared about 0.39 (test 0.38); typical error factor about 1.7 to 1.9.
- **Logistic regression:** high vs low salary, accuracy about 80%, AUC about 0.88.
- **Limitations:** 263 labelled players, mild heteroscedasticity, multicollinearity, and factors outside the data.